



5장 회귀

2조 고은비 백재은 장서연

a파트 목차

#01 / 01_회귀 소개 ~ 03_경사 하강법

#02 / 04_(실습) 보스턴 주택 가격 예측 ~ 05_ 다항회귀

#03 / 06_규제 선형 회귀 ~ 08_회귀 트리



01. 회귀 소개



#01_01 용어 정리

회귀 (Regression)

-> 여러 개의 독립변수와 한 개의 종속변수 간의 상관관계를 수치적으로 모델링하는 분석 기법

갈튼(Galton)의 유전 연구에서 유래 => 자식의 키는 부모 평균에 회귀한다~?

통계적 관점에서 회귀

선형 회귀식) $Y = w_1 * X_1 + w_2 * X_2 + \dots + w_n * X_n$

w = 회귀 계수 (coefficient), X = 입력값, Y = 예측값

↳ $w_1, w_2, \dots, w_n$ = 독립변수값에 영향을 미치는 회귀 계수 (Regression coefficient)

Ex) 아파트 가격 예측

독립변수 = 아파트 방 개수, 방 크기, 주변 학군 ... / 종속변수 = 아파트 가격

#01_01 용어 정리

머신러닝 관점에서 회귀

독립변수 = 피처 / 종속변수 = 결정 값

머신러닝 회귀 예측의 핵심

주어진 피처와 결정 값 데이터 기반 학습 => 최적의 회귀 계수 찾아내는 것

회귀 유형 구분

독립변수 개수	회귀 계수의 결합
1개: 단일 회귀	선형: 선형 회귀
여러 개: 다중 회귀	비선형: 비선형 회귀

〈 회귀 유형 구분 〉

지도학습의 유형

Classification



Category 값
(이산값)

Regression



숫자값
(연속값)

〈 분류와 회귀의 예측 결과값 차이 〉

#01_01 용어 정리

선형 회귀

- 여러 가지 회귀 중 가장 많이 사용
- 실제 값과 예측값의 차이 (오류의 제곱 값)를 최소화하는 직선형 회귀선을 최적화하는 방식
- 규제 (Regularization) 방법에 따라 다시 유형 분류
 - ↳ 일반적인 선형 회귀의 과적합 문제를 해결하기 위해서 회귀 계수에 페널티 값을 적용하는 것

일반 선형 회귀	예측값과 실제 값의 RSS (Residual Sum of Squares)를 최소화할 수 있도록 회귀 계수를 최적화 규제(Regularization)를 적용 X
릿지 (Ridge)	선형 회귀에 L2 규제를 추가한 회귀 모델 L2 규제: 상대적으로 큰 회귀 계수 값의 예측 영향도를 감소시키기 위해서 회귀 계수값을 더 작게 만드는 규제
라쏘 (Lasso)	선형 회귀에 L1 규제를 적용한 방식 L1 규제: 예측 영향력이 작은 피처의 회귀 계수를 0으로 만들어 회귀 예측 시 피처가 선택되지 않게 하는 것 (= 피처 선택 기능)
엘라스틱넷 (ElasticNet)	L2, L1 규제를 함께 결합한 모델 주로 피처가 많은 데이터 세트에 적용, L1 규제로 피처의 개수를 줄임과 동시에 L2 규제로 계수 값의 크기 조정
로지스틱 회귀 (Logistic Regression)	분류에 사용되는 선형 모델, 매우 강력한 분류 알고리즘 일반적으로 이진 분류뿐만 아니라 최소 영역의 분류, 예를 들어 텍스트 분류와 같은 영역에서 뛰어난 예측 성능

02. 단순 선형 회귀



#02_01 단순 선형 회귀를 통한 회귀 이해

#1 단일 입력 변수 -> 단순 선형 회귀

주택 가격이 주택의 크기만으로 결정된다 가정
일반적으로 주택의 크기와 가격 양의 관계
=> 선형(직선의 관계)

회귀 모델: $f(x) = w_0 + w_1 * x$
-> 기울기 w_0 , 절편(intercept) w_1 회귀 계수

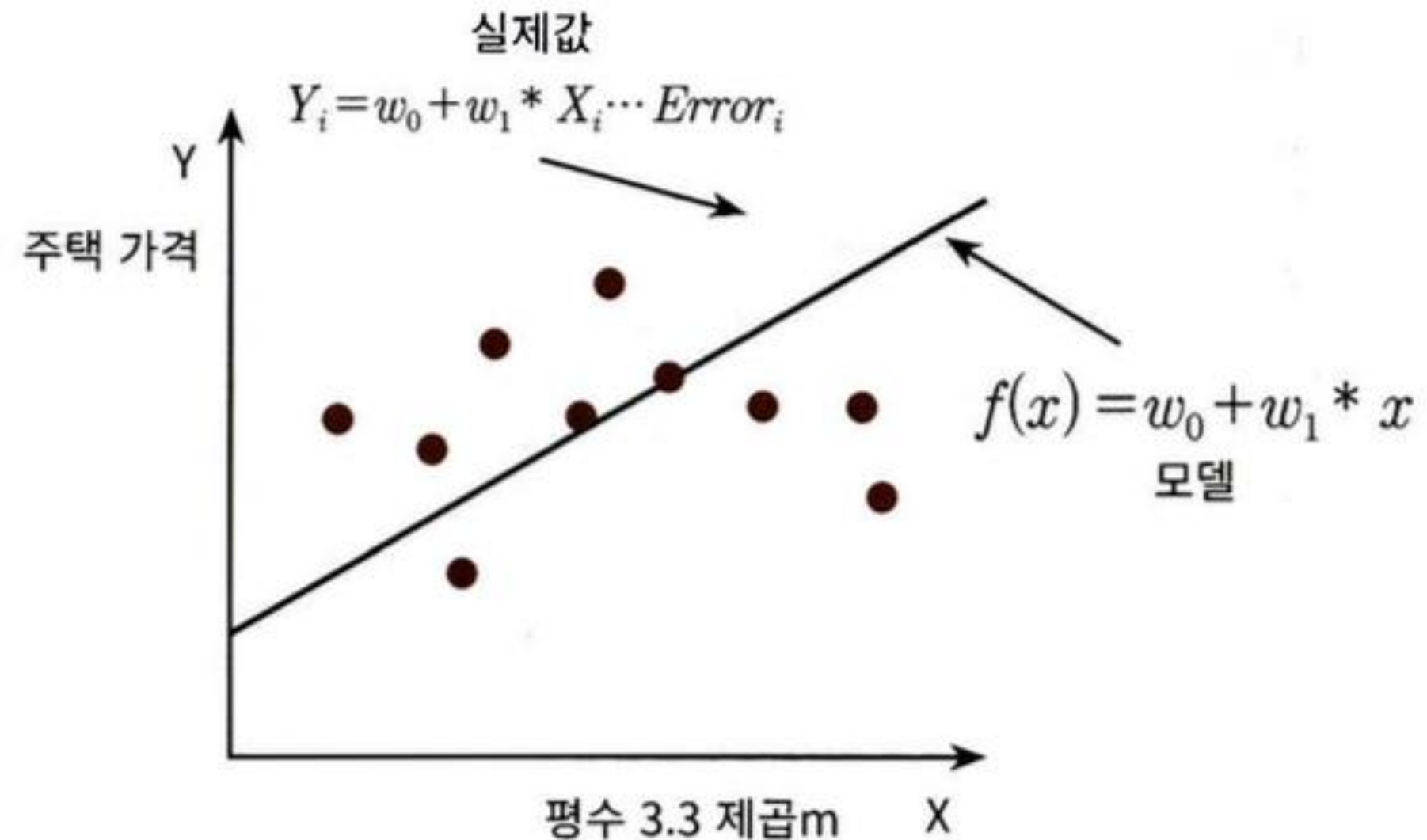
실제 주택 가격: $w_0 + w_1 * x + \text{오류 값}$

↑
남은 오류, 잔차

최적의 회귀 모델

=> 전체 데이터의 잔차 합이 최소가 되는 모델

=> 오류 값 합이 최소가 될 수 있는 회귀 계수 찾기

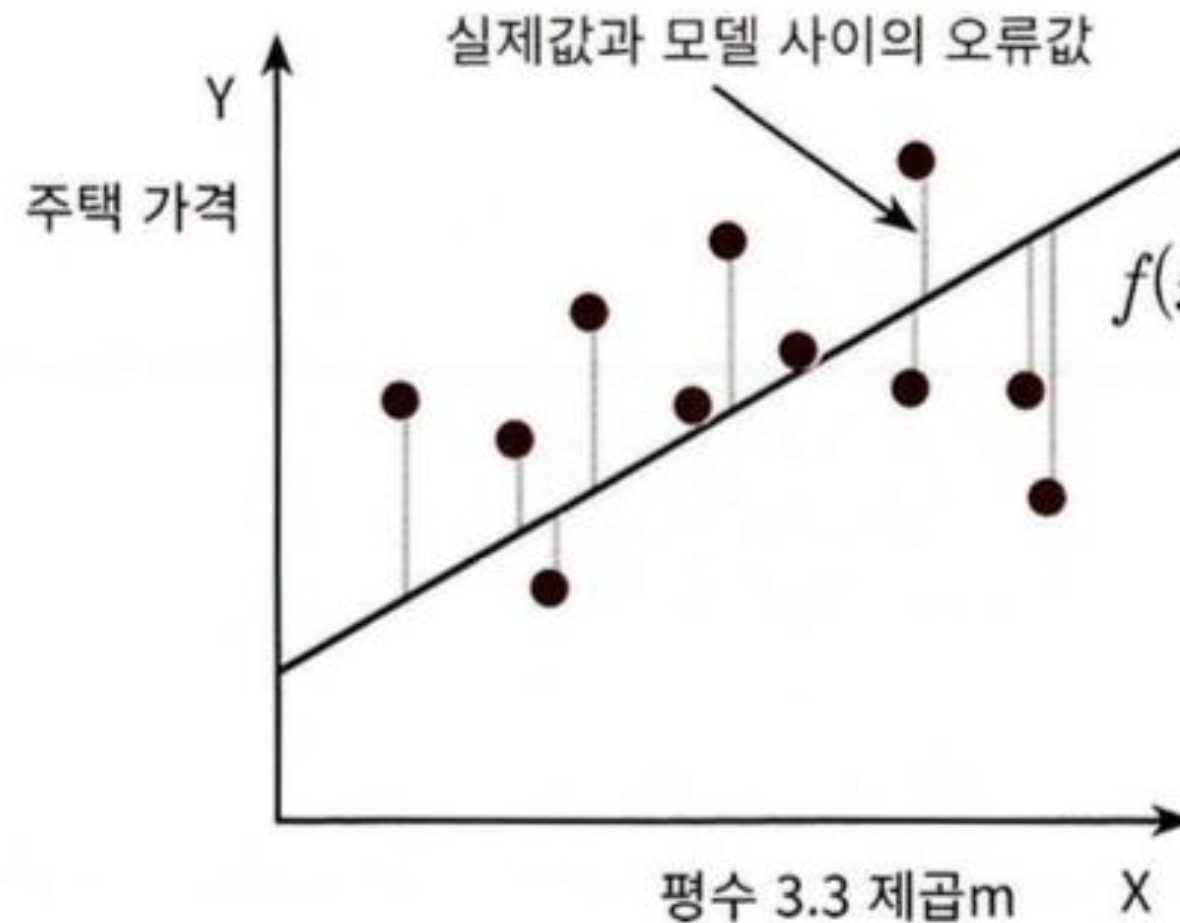


#02_02 단순 선형 회귀를 통한 회귀 이해

#2 오류 값

오류 값 => 양/음 가능! ... 오류 합 계산 시 1) 절댓값 (Mean Absolute Error)
2) 오류 값의 제곱 (RSS, Residual Sum of Square)

즉! $\text{Error}^2 = \text{RSS}$ (미분 계산 easy)



$$f(x) = w_0 + w_1 * x$$

$$\begin{aligned} \text{RSS} = & (\#1 \text{ 주택 가격} - (w_0 + w_1 \#1 \text{ 주택크기}))^2 \\ & + (\#2 \text{ 주택 가격} - (w_0 + w_1 \#2 \text{ 주택크기}))^2 \\ & + (\#3 \text{ 주택 가격} - (w_0 + w_1 \#3 \text{ 주택크기}))^2 \\ & + \dots (\text{모든 학습 데이터에 대해 RSS 수행}) \end{aligned}$$

#02_03 단순 선형 회귀를 통한 회귀 이해

#3 RSS

RSS -> 변수가 w_0, w_1 인 식으로 표현

머신러닝 기반 회귀의 핵심 -> RSS를 최소로 하는 w_0, w_1 (회귀 계수)를 학습을 통해 찾는 것

*학습 데이터 상 독립변수(X)와 종속변수(Y)는 모두 상수로 간주

$$RSS(w_0, w_1) = \frac{1}{N} \sum_{i=1}^N (y_i - (w_0 + w_1 * x_i))^2$$

비용 함수
손실 함수 (loss function)

(i는 1부터 학습 데이터의 총 건수 N까지)

↑ ↑

비용 (Cost) 회귀 계수

머신러닝 회귀 알고리즘 -> 데이터 학습을 통해 비용 함수의 반환 값 (오류 값) 지속해서 감소시켜,
최종적으로 더이상 감소하지 않는 최소의 오류 값 도출

03. 경사 하강법



#03_01 경사 하강법 Intro...

경사 하강법의 필요성

비용 함수(RSS)를 최소화하는 W 파라미터 값을 구하는 게 머신러닝 회귀의 목표!
파라미터 개수 적을 때 -> 고차원 방정식으로 직접 가능
But! 👉 파라미터가 많아지면 계산 복잡, 방대

경사 하강법의 역할

비용 함수(RSS) 를 반복적으로 줄이는 방향으로 W 값을 업데이트하는 수치 최적화 기법
머신러닝의 핵심 원리: "데이터를 기반으로 스스로 학습한다"를 가능케 하는 핵심 알고리즘

개념 in 경사 하강법

'Gradient Descent'

- ▶ '점진적인 하강(경사)'이라는 의미
- ▶ 오차를 줄이는 방향으로 W 값을 계속 업데이트

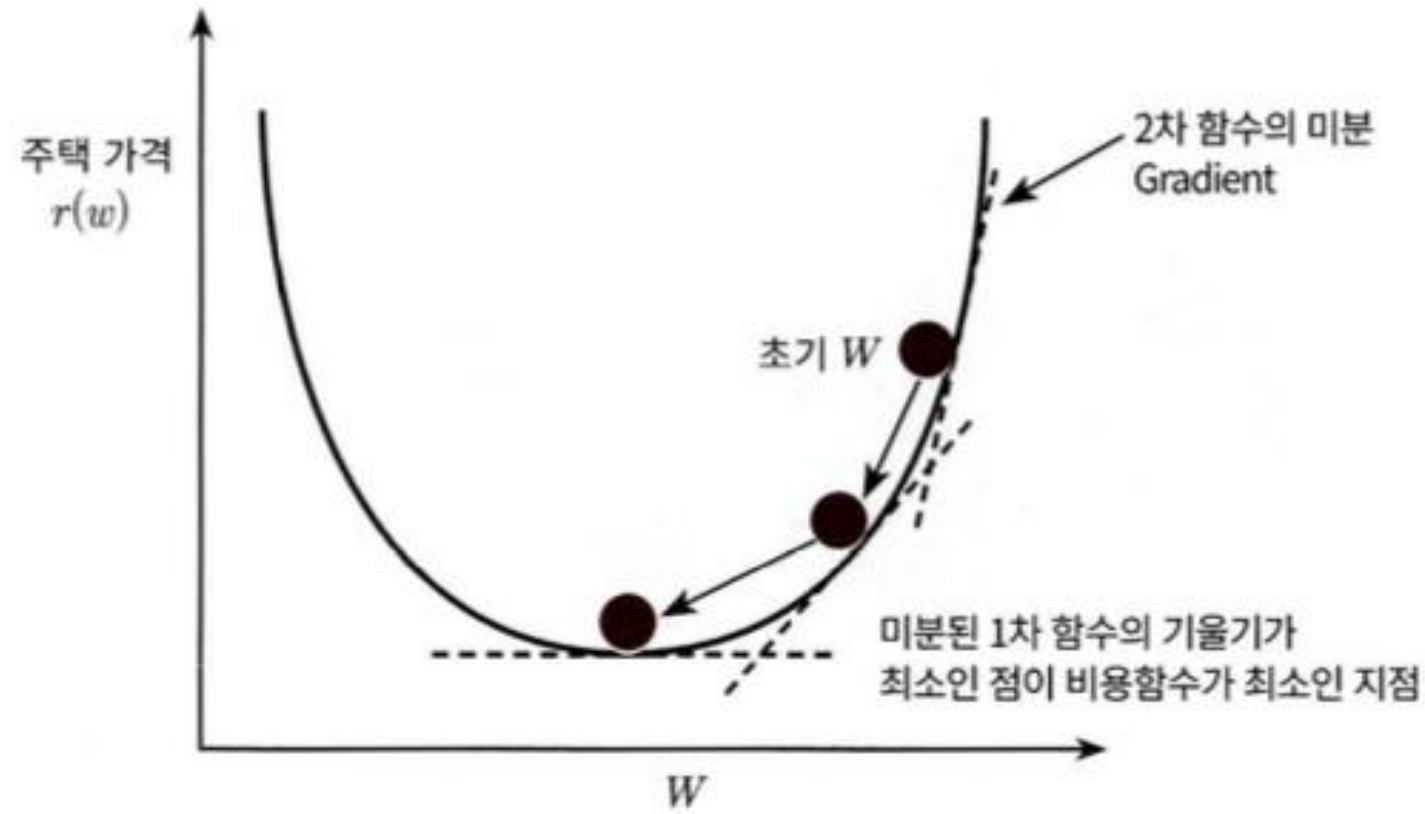
#03_02 핵심 개념

반복적으로 비용 함수의 반환 값
= 예측값과 실제 값의 차이가 작아지는 방향성
-> w 파라미터를 지속해서 보정

오류 값이 더이상 작아지지 않을 때 = 최소 cost
= 최적 파라미터



#03_03 경사 하강법 그래프 및 수식



오류 값이 더이상 작아지지 않을 때
= 최소 cost
= 최적 파라미터

$$R(w) = \frac{1}{N} \sum_{i=1}^N (y_i - (w_0 + w_1 * x_i))^2$$
$$\frac{\partial R(w)}{\partial w_1} = \frac{2}{N} \sum_{i=1}^N -x_i * (y_i - (w_0 + w_1 x_i)) = -\frac{2}{N} \sum_{i=1}^N x_i * (\text{실제값}_i - \text{예측값}_i)$$
$$\frac{\partial R(w)}{\partial w_0} = \frac{2}{N} \sum_{i=1}^N -(y_i - (w_0 + w_1 x_i)) = -\frac{2}{N} \sum_{i=1}^N (\text{실제값}_i - \text{예측값}_i)$$

#03_04 경사 하강법 구현

👉 회귀식 $y = 4X + 6$ 을 근사하기 위한
100개의 데이터 세트

-> 경사 하강법을 이용 해 회귀 계수 w_1, w_0 도출

$w_1 = 4$ (기울기), $w_0 = 6$ (절편값) 근사
임의의 값은 노이즈를 위해 만듦

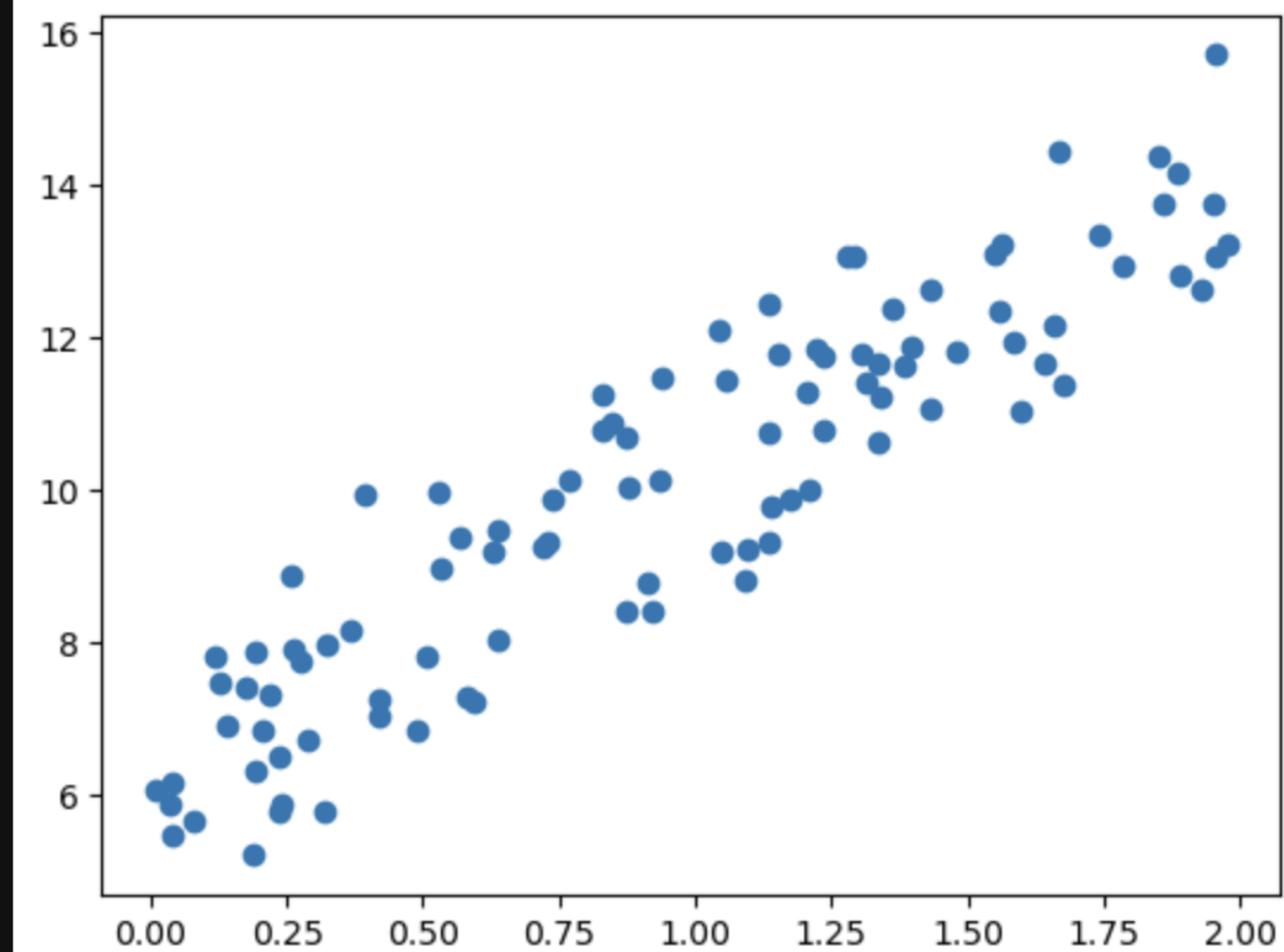
산점도로 시각화

```
[1]: import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

np.random.seed(0)

X = 2 * np.random.rand(100, 1)
y = 6 + 4 * X + np.random.randn(100, 1)

plt.scatter(X, y)
plt.show()
```



#03_04 경사 하강법 구현

👉 비용함수 `get_cost()`
정의

```
[3]: def get_cost(y, y_pred):  
      N = len(y)  
      cost = np.sum(np.square(y - y_pred)) / N  
      return cost
```

👉 `get_weight_updates()`
-> 입력 배열 `X`값에 대한
예측 배열 `y_pred` 구함
= 입력 배열 `X`와 `w1`배열의
내적
-> numpy 내적 연산 `dot()`
이용

```
[5]: def get_weight_updates(w1, w0, X, y, learning_rate=0.01):  
      N = len(y)  
  
      # 초기화  
      w1_update = np.zeros_like(w1)  
      w0_update = np.zeros_like(w0)  
  
      # 예측 배열 계산, 예측과 실제 값 차이 계산  
      y_pred = np.dot(X, w1.T) + w0  
      diff = y - y_pred  
  
      # 모두 1값을 가진 행렬 생성  
      w0_factors = np.ones((N, 1))  
  
      # 업데이트 계산  
      w1_update = -(2/N)*learning_rate*(np.dot(X.T, diff))  
      w0_update = -(2/N)*learning_rate*(np.dot(w0_factors.T, diff))  
  
      return w1_update, w0_update
```

-> `w1, w0` 업데이트 값을
행렬 연산하여 반환

#03_04 경사 하강법 구현

👉 gradient_descent_steps()

Get_weight_updates() 를 경사
하강 방식으로 반복 수행, w1, w0
업데이트

```
[7]: # 입력 인자 iters로 주어진 횟수만큼 반복적으로 w1과 w0를 업데이트
def gradient_descent_steps(X, y, iters=1000):
    # w0와 w1을 모두 0으로 초기화
    w0 = np.zeros((1, 1))
    w1 = np.zeros((1, 1))

    # 인자로 주어진 iters 만큼 반복적으로 get_weight_updates() 호출하여 w1, w0 업데이트 수행.
    for ind in range(iters):
        w1_update, w0_update = get_weight_updates(w1, w0, X, y, learning_rate=0.01)
        w1 = w1 - w1_update
        w0 = w0 - w0_update
    return w1, w0
```

👉 get_cost()

예측값과 실제값의 RSS 차이 계산
-> 예측 오류 계산

Output

L1: w1, w0 호출

L2: 예측 오류 계산

```
[9]: def get_cost(y, y_pred):
    N = len(y)
    cost = np.sum(np.square(y - y_pred))/N
    return cost

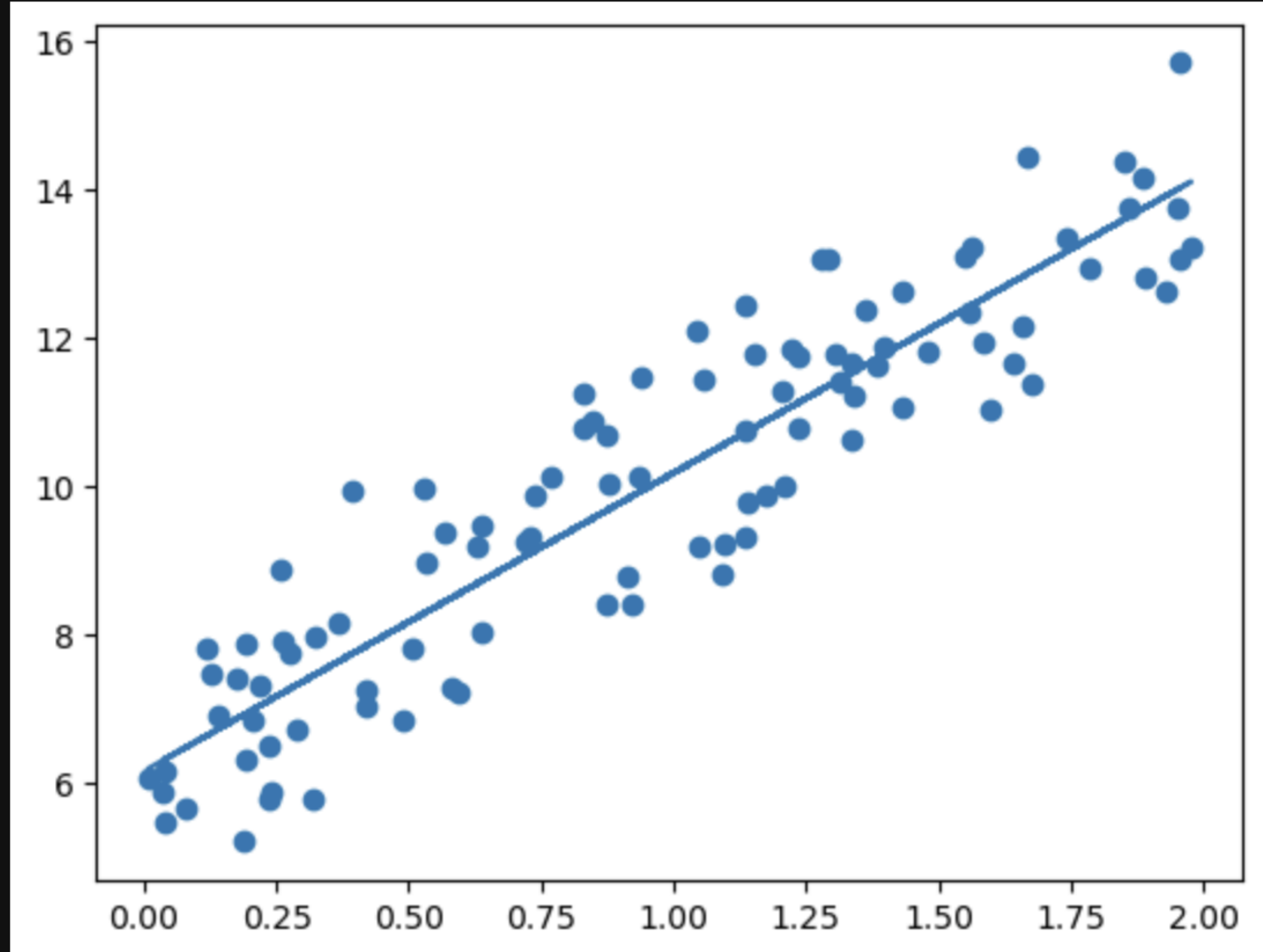
w1, w0 = gradient_descent_steps(X, y, iters=1000)
print("w1:{0:.3f} w0:{1:.3f}".format(w1[0, 0], w0[0, 0]))
y_pred = w1[0, 0] * X + w0
print('Gradient Descent Total Cost:{0:.4f}'.format(get_cost(y, y_pred)))

w1:4.022 w0:6.162
Gradient Descent Total Cost:0.9935
```

#03_04 경사 하강법 구현

👉 y_pred 기반한 회귀선 시각화

```
[11]: plt.scatter(X, y)  
plt.plot(X, y_pred)  
plt.show()
```



#03_05 확률적 경사 하강법 구현

- 👉 경사 하강법 = 모든 데이터에 대해 업데이트 -> 오래 걸림!
=> 확률적 경사 하강법 (Stochastic Gradient Descent) 이용 or 미니 배치
전체 입력 데이터로 w 가 업데이트되는 값 계산 X, 일부 데이터만! = 빠르다!!

👉 stochastic_gradient_descent_steps()

확률적 경사 하강법 구현
X, y 데이터에서 랜덤하게 batch_size
만큼 데이터 추출 후 해당 데이터 기반
업데이트 데이터 계산하는 부분에서
기존 구현과 차별점!

```
[13]: def stochastic_gradient_descent_steps(X, y, batch_size = 10, iters = 1000):  
    w0 = np.zeros((1,1))  
    w1 = np.zeros((1, 1))  
  
    for ind in range(iters):  
        np.random.seed(ind)  
  
        # 전체 X, y 데이터에서 랜덤하게 batch_size만큼 데이터 추출, 샘플로 저장  
  
        stochastic_random_index = np.random.permutation(X.shape[0])  
        sample_X = X[stochastic_random_index[0:batch_size]]  
        sample_y = y[stochastic_random_index[0:batch_size]]  
  
        # 랜덤하게 batch_size 만큼 추출된 데이터 기반으로 계산 후 업데이트  
  
        w1_update, w0_update = get_weight_updates(w1, w0, sample_X,  
                                                    sample_y, learning_rate = 0.01)  
  
        w1 = w1 - w1_update  
        w0 = w0 - w0_update  
  
    return w1, w0
```

#03_05 확률적 경사 하강법 구현

```
[15]: w1, w0 = stochastic_gradient_descent_steps(X, y, iters=1000)
      print("w1:", round(w1[0,0], 3), "w0:", round(w0[0,0],3))
      y_pred = w1[0,0] * X + w0
      print('Stochastic Gradient Descent Total Cost:{0:.4f}'.format(get_cost(y, y_pred)))

w1: 4.028 w0: 6.156
Stochastic Gradient Descent Total Cost:0.9937
```

👉 w1, w0 & 예측 오류 비용 계산 (= Output)

```
w1:4.022 w0:6.162
Gradient Descent Total Cost:0.9935
```

👉 기존 경사 하강법으로 구한 값과 차이 적음 = 예측 성능상 차이가 거의 없음
따라서 데이터가 방대할 때는 주로 확률적 경사 하강법 활용

#03_06 피처가 여러 개일 때 회귀 계수 도출 방법

- 👉 피처가 한 개인 경우 예측값 $\hat{Y} = w_0 + w_1 * X$
- 👉 피처가 M개 있다면 회귀 계수도 M+1 개
- 회귀식: $\hat{Y} = w_0 + w_1 * X_1 + w_2 * X_2 + \dots + w_{100} * X_{100}$
(X1 ~ X100 이라 가정)



- 👉 예측 행렬 y_pred 처럼 행렬 dot product 로 구하기
- $\hat{Y} = \text{np.dot}(X_{mat}, W^T) + w_0$

$$\hat{Y} \quad X_{mat}$$

	Feature 1	Feature 2	...	Feature M
y_1	x_{11}	x_{12}	$\dots$	x_{1m}
y_2	x_{21}	x_{22}	$\dots$	x_{2m}
$\dots$	$\dots$	$\dots$	$\dots$	$\dots$
y_n	x_{n1}	x_{n2}	$\dots$	x_{nm}

$\star$ 내적 $[w_1 \ w_2 \ \dots \ w_m]^T + w_0$

- 👉 w0을 W 배열 안에 포함시키기 위해 Xmat의 가장 처음 열에 모든 데이터 값이 1인 피처 Feat 0 추가

최종 회귀 예측값 수식 도출

$$\hat{Y} = X_{mat} * W^T$$

$\hat{Y}$ 1값을 가진 피처 추가 X_{mat}

	Feat 0	Feat 1	Feat 2	...	Feat M
y_1	1	x_{11}	x_{12}	$\dots$	x_{1m}
y_2	1	x_{21}	x_{22}	$\dots$	x_{2m}
$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$
y_n	1	x_{n1}	x_{n2}	$\dots$	x_{nm}

w_0 을 W 배열 내에 포함

$\star$ 내적 $[w_0 \ w_1 \ w_2 \ \dots \ w_m]^T$

$$\hat{Y} = X_{mat} * W^T$$

b파트 목차

#5-4. 실습 – 보스턴 주택 가격 예측

#5-5. 다항 회귀



#5-4. 실습 – 보스턴 주택 가격 예측



LinearRegression 클래스 - Ordinary Least Squares

✓ 용어 정리

$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_i x_{ip} + \epsilon_i$ > 기본적 선형회귀식, β 들이 회귀계수

$\hat{\beta}_0$ $\hat{y}_i$ $\hat{\beta}_1$ > 예측값, fitted value, estimate

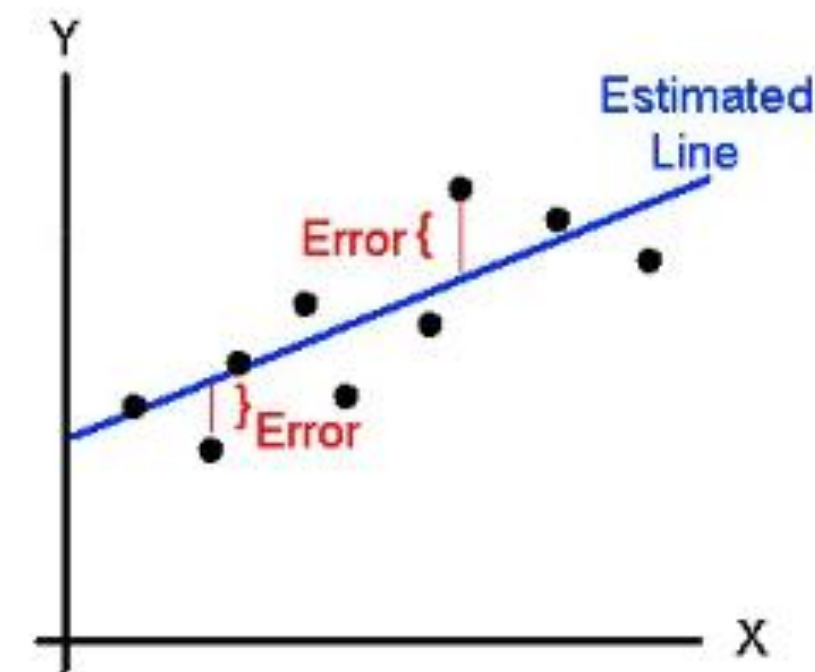
Estimated (or predicted) Y value for observation i

Estimate of the regression intercept

Estimate of the regression slope

Value of X for observation i

$$\hat{Y}_i = b_0 + b_1 X_i$$



LinearRegression 클래스 - Ordinary Least Squares

```
sklearn.linear_model.LinearRegression(fit_intercept = True, normalize = False,  
                                       copy_X = True, n_jobs = 1)
```

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip} + \epsilon_i$$

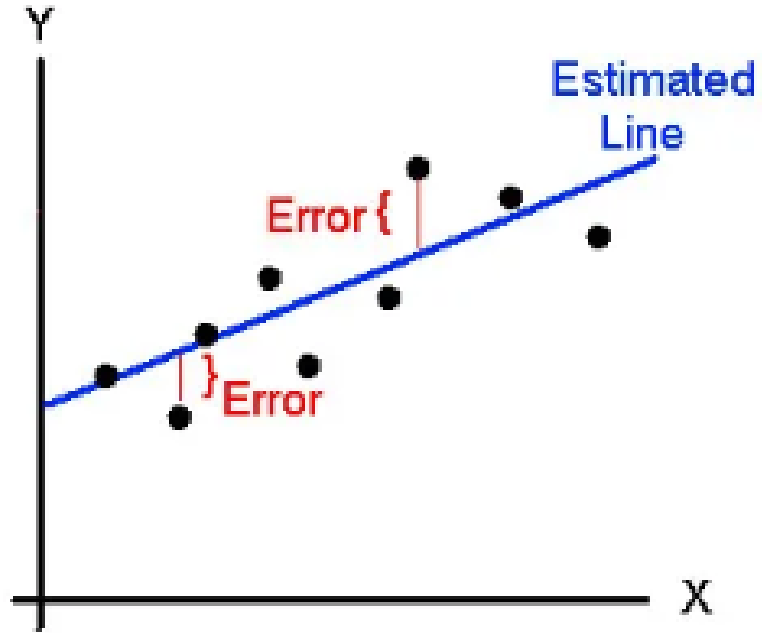
fit_intercept : $\hat{\beta}_0$ 구할지 말지 결정,
default = True, False로 설정시 0으로 지정됨

normalize : 회귀 수행전 입력 데이터셋(x_{ip})를 정규화할지 말지 결정
default = False

coef : $\hat{\beta}$ 들을 모아놓은 배열 (intercept 제외)

intercept_ : intercept 값

회귀 평가 지표



✓ 최소화하고 싶은 **Error** 평균값

$$\frac{1}{n} \sum (y_i - \hat{y}_i)$$

$$MAE = \frac{1}{n} \sum_{i=1}^n |Y_i - \hat{Y}_i|$$

❶ 각 값에 절댓값을 취하면 MAE
metrics.mean_absolute_error / GridSearchCV : 'neg_mean_absolute_error'

$$MSE = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

❷ 각 값에 제곱을 취하면 MSE
metrics.mean_squared_error / GridSearchCV : 'neg_mean_squared_error'

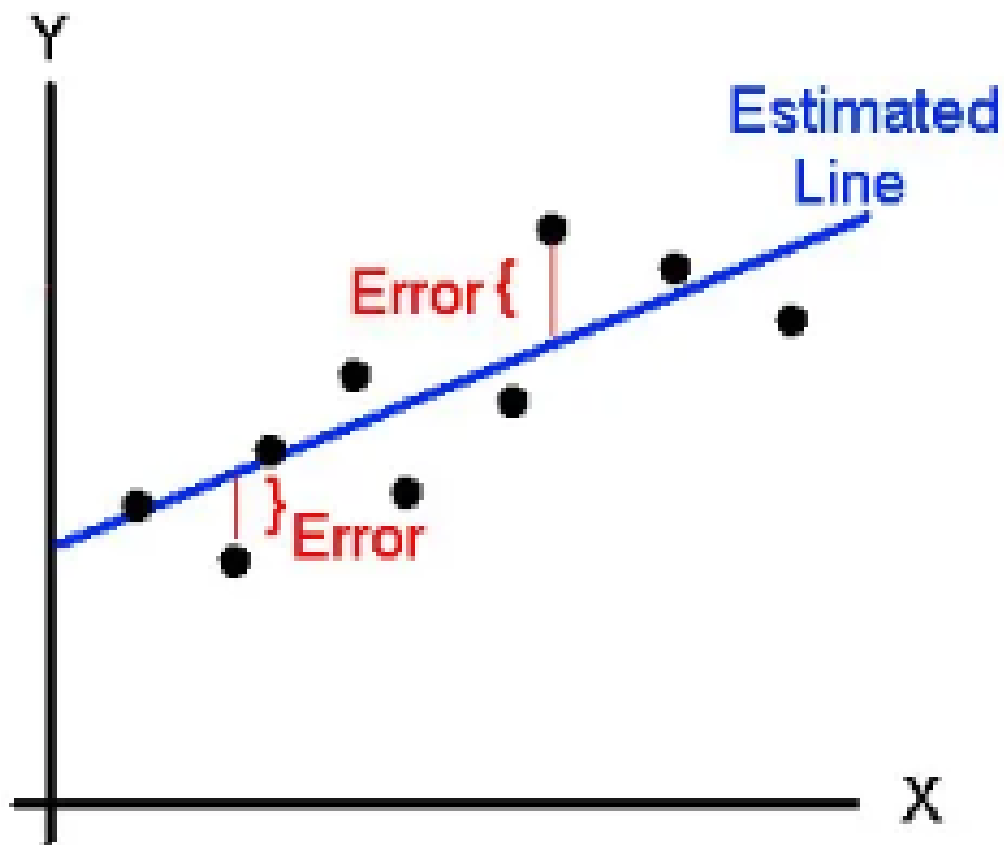
$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2}$$

❸ MSE에 루트를 취하면 RMSE
MSE api + squared = False / GridSearchCV : 'neg_root_mean_squared_error'

$$MSLE = \frac{1}{n} \sum (\log(1 + y_i) - \log(1 + \hat{y}_i))^2$$

❹ 각 값에 로그를 취하면 MSLE
metrics.mean_squared_log_error / GridSearchCV : 'neg_root_mean_squared_error'

회귀 평가 지표



④ R^2 결정계수 (Coefficient of Determination)

파머완 : 예측값 variance / 실제값 variance

Error 값이 작을수록 1에 가까워지는 값 > 1에 가까울 수록 좋은 예측
metrics.r2_score/ GridSearchCV : 'r2'

$$1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

LinearRegression을 이용해 보스턴 주택 가격 회귀 구현

✅ 데이터셋 로드

› 윤리적 문제로 사이킷런 최신 버전에서는 load_boston() 지원 ❌

```
import pandas as pd
import numpy as np

boston = pd.read_csv("http://lib.stat.cmu.edu/datasets/boston", delim_whitespace=True, skiprows=22, header=None)
data = np.hstack([raw_df.values[::2, :], raw_df.values[1::2, :2]])
target = raw_df.values[1::2, 2]

# 컬럼 이름
columns = [
    "CRIM", "ZN", "INDUS", "CHAS", "NOX", "RM", "AGE",
    "DIS", "RAD", "TAX", "PTRATIO", "B", "LSTAT"
]

# DataFrame 만들기
boston_df = pd.DataFrame(data, columns=columns)
boston_df["PRICE"] = target

# 확인
display(boston_df)
```

LinearRegression을 이용해 보스턴 주택 가격 회귀 구현

✓ 데이터셋 설명

> 506행, 14열(13 X + 1 Y) / 결측치 없음

$$\hat{y} = \beta_0 + \beta_1x_1 + \beta_2x_2 + \cdots + \beta_{13}x_{13}$$

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	PRICE
0	0.00632	18.0	2.31	0.0	0.538	6.575	65.2	4.0900	1.0	296.0	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0.0	0.469	6.421	78.9	4.9671	2.0	242.0	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0.0	0.469	7.185	61.1	4.9671	2.0	242.0	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0.0	0.458	6.998	45.8	6.0622	3.0	222.0	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0.0	0.458	7.147	54.2	6.0622	3.0	222.0	18.7	396.90	5.33	36.2
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0.0	0.573	6.593	69.1	2.4786	1.0	273.0	21.0	391.99	9.67	22.4
502	0.04527	0.0	11.93	0.0	0.573	6.120	76.7	2.2875	1.0	273.0	21.0	396.90	9.08	20.6
503	0.06076	0.0	11.93	0.0	0.573	6.976	91.0	2.1675	1.0	273.0	21.0	396.90	5.64	23.9
504	0.10959	0.0	11.93	0.0	0.573	6.794	89.3	2.3889	1.0	273.0	21.0	393.45	6.48	22.0
505	0.04741	0.0	11.93	0.0	0.573	6.030	80.8	2.5050	1.0	273.0	21.0	396.90	7.88	11.9

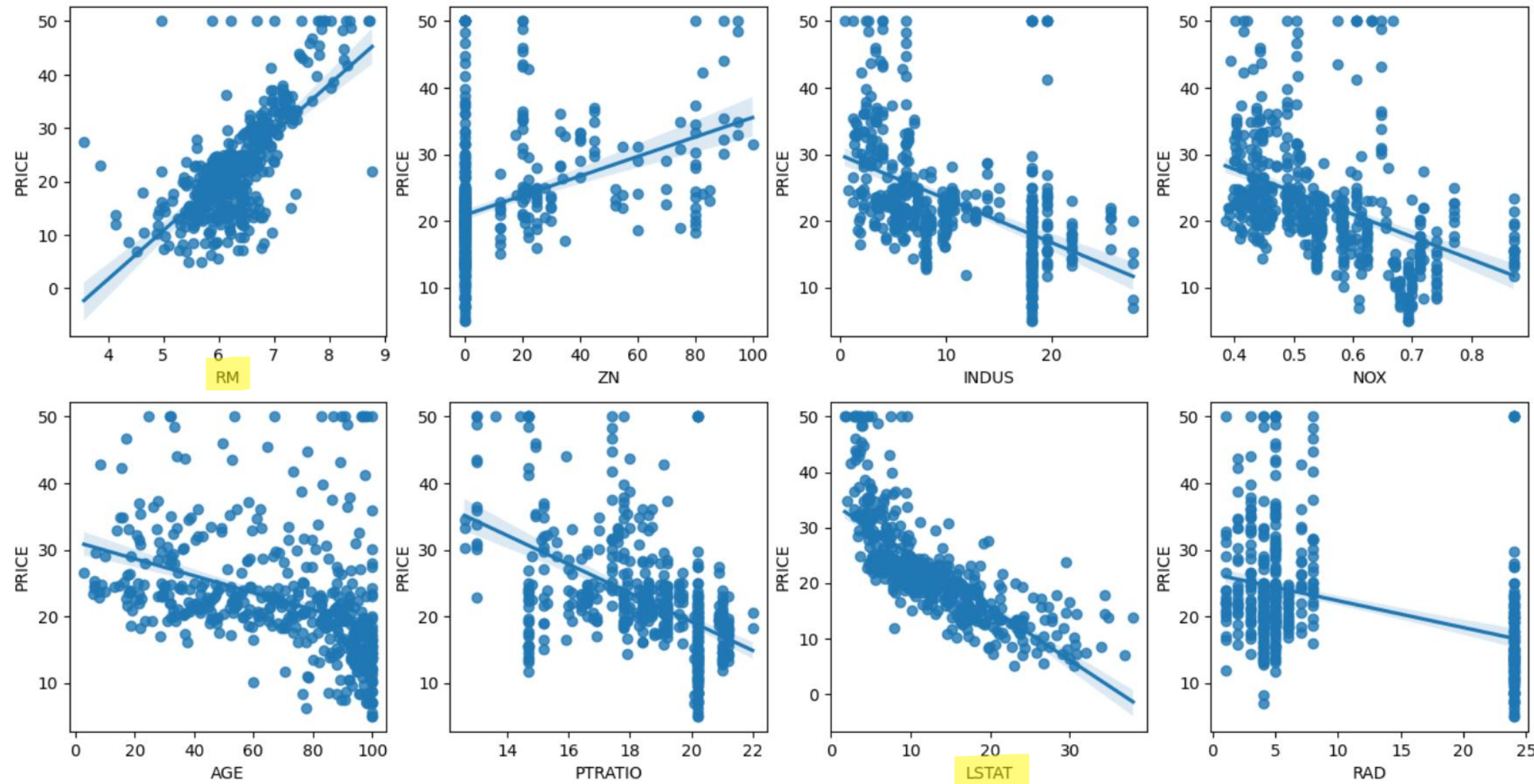
506 rows × 14 columns

- CRIM: 지역별 범죄 발생률
- ZN: 25,000평방피트를 초과하는 거주 지역의 비율
- INDUS: 비상업 지역 넓이 비율
- CHAS: 찰스강에 대한 더미 변수(강의 경계에 위치한 경우는 1, 아니면 0)
- NOX: 일산화질소 농도
- RM: 거주할 수 있는 방 개수
- AGE: 1940년 이전에 건축된 소유 주택의 비율
- DIS: 5개 주요 고용센터까지의 가중 거리
- RAD: 고속도로 접근 용이도
- TAX: 10,000달러당 재산세율
- PTRATIO: 지역의 교사와 학생 수 비율
- B: 지역의 흑인 거주 비율
- LSTAT: 하위 계층의 비율
- MEDV: 본인 소유의 주택 가격(중앙값)

LinearRegression을 이용해 보스턴 주택 가격 회귀 구현

✓ EDA – 데이터들의 상관관계 시각화

```
fig,axs = plt.subplots(figsize = (16,8), ncols=4, nrows =2)
lm_features = ['RM','ZN','INDUS','NOX','AGE','PTRATIO','LSTAT','RAD']
for i, feature in enumerate(lm_features):
    row = int(i/4)
    col = i%4
    sns.regplot(x=feature,y='PRICE',data=boston,ax=axs[row][col])
```



▶ RM은 양의 상관관계, LSTAT는 음의 상관관계가 두드러짐.

LinearRegression을 이용해 보스턴 주택 가격 회귀 구현

✓ 각 회귀계수에 대한 평가지표값 구하기 – MSE, R2score

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```
y_target = boston_df['PRICE']
X_data = boston_df.drop(['PRICE'], axis=1, inplace=False)

X_train, X_test, y_train, y_test = train_test_split(X_data, y_target, test_size = 0.3,
                                                    random_state = 156)

lr = LinearRegression()
lr.fit(X_train, y_train)

y_preds = lr.predict(X_test)
mse = mean_squared_error(y_test, y_preds)
rmse = np.sqrt(mse)

print('MSE:{0:.3f}, RMS:{1:.3F}'.format(mse, rmse))
print('R2 score: {0:.3f}'.format(r2_score(y_test, y_preds)))
```

MSE: 17.297, RMS: 4.159

R2 score: 0.757

LinearRegression을 이용해 보스턴 주택 가격 회귀 구현

✓ 예측된 $\hat{y}$ 값들 확인하기 - model.intercept, model.coef_

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_{13} x_{13}$$

```
print('절편값:', lr.intercept_)
print('회귀 계수값:', np.round(lr.coef_, 1))
```

절편값: 40.995595172164826
회귀 계수값: [-0.1 0.1 0. 3. -19.8 3.4 0. -1.7 0.4 -0. -0.9 0. -0.6]

↓ Series 변환, 크기 순으로 정렬

```
coeff = pd.Series(data = np.round(lr.coef_, 1), index = X_data.columns)
coeff.sort_values(ascending=False)
```

	0
RM	3.4
CHAS	3.0
RAD	0.4
ZN	0.1
INDUS	0.0
B	0.0
TAX	-0.0
AGE	0.0
CRIM	-0.1
LSTAT	-0.6
PTRATIO	-0.9
DIS	-1.7
NOX	-19.8

LinearRegression을 이용해 보스턴 주택 가격 회귀 구현

✓ 교차검증하기 – cross_val_score()

Scoring = '평가지표' 로 설정하면 반환되는 수치값은 (-)값이므로,
-를 곱해 양수로 만들어야 원래 구하고 싶었던 MSE값이 됨.
> 사이킷런의 지표값 평가 기준은 높은 지표값일 수록 좋은 모델.

```
from sklearn.model_selection import cross_val_score

y_target = boston_df['PRICE']
X_data = boston_df.drop(['PRICE'],axis =1, inplace = False)
lr = LinearRegression()

neg_mse_scores = cross_val_score(lr,X_data,y_target,scoring="neg_mean_squared_error",cv=5)
rmse_scores = np.sqrt(-1*neg_mse_scores)
avg_rmse = np.mean(rmse_scores)

print('5 folds 의 개별 Negative MSE scores:', np.round(neg_mse_scores,2))
print('5 folds 의 개별 RMSE scores:', np.round(rmse_scores,2))
print('5 folds 의 평균 RMSE: {0:.3f}'.format(avg_rmse))
```

5 folds 의 개별 Negative MSE scores: [-12.46 -26.05 -33.07 -80.76 -33.31]

5 folds 의 개별 RMSE scores: [3.53 5.1 5.75 8.99 5.77]

5 folds 의 평균 RMSE: 5.829

#5-5. 다항 회귀



#5-5. 다항회귀

✓ 다항회귀 – 선형회귀

사이킷런 > 비선형 함수를 선형 모델에 적용

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_{13} x_{13}$$

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \epsilon$$

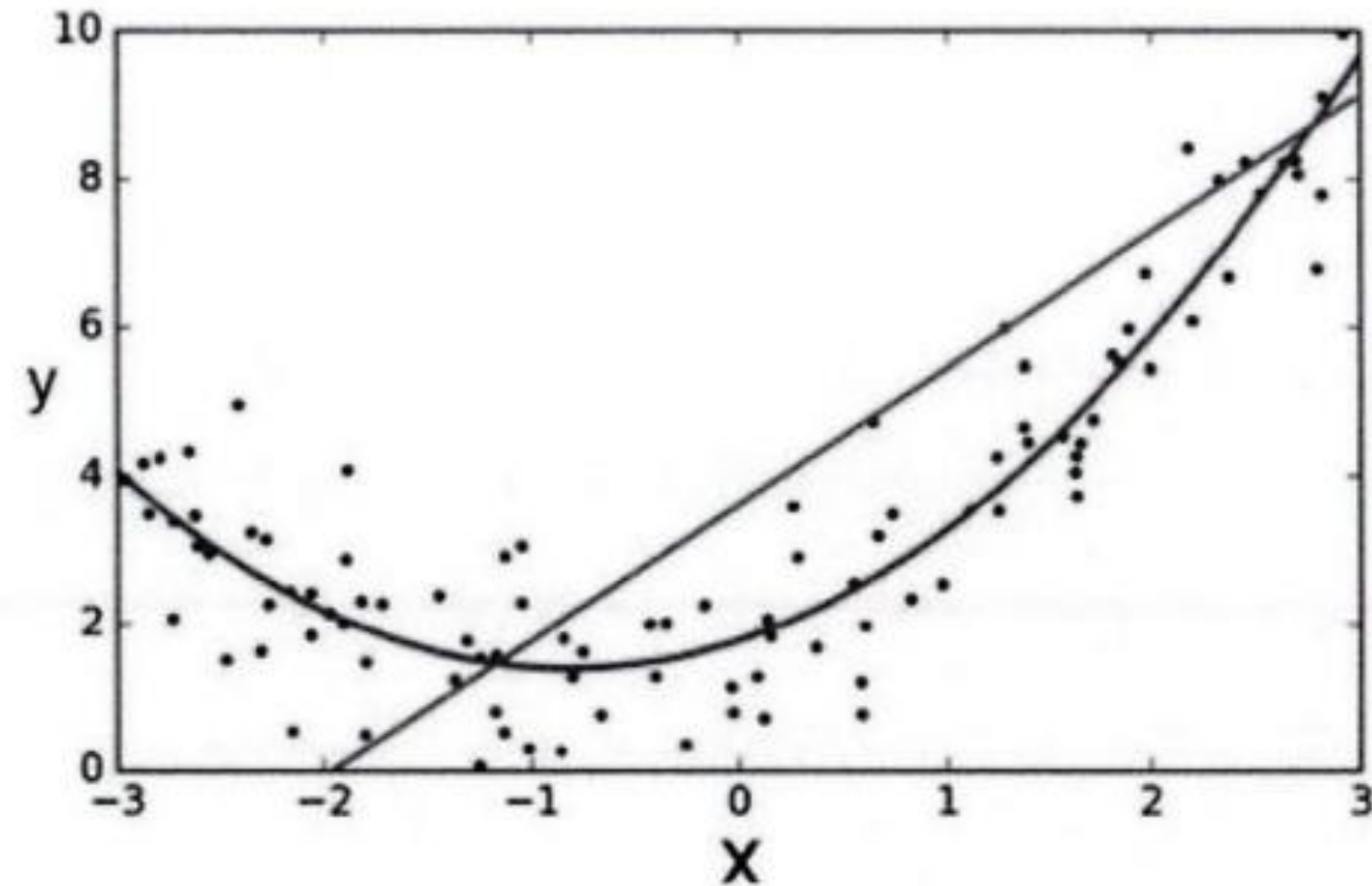
$$y = \beta_0 + \beta_1 z_1 + \beta_2 z_2 + \beta_3 z_3 + \cdots + \epsilon$$

$$z_1 = x$$

$$z_2 = x^2$$

$$z_3 = x^3$$

> β 들에 대해 식이 선형



〈 주어진 데이터 세트에서 다항 회귀가 더 효과적임 〉

#5-5. 다항회귀

✓ 사이킷런에서의 2차 다항회귀

- › sklearn.preprocessing의 PolynomialFeatures 사용, feature를 다항식 피처로 변환

```
from sklearn.preprocessing import PolynomialFeatures
import numpy as np

# 다항식으로 변환한 단항식 생성, [[0,1],[2,3]]의 2X2 행렬 생성
X = np.arange(4).reshape(2,2)
print('일차 단항식 계수 feature:\n', X )

# degree = 2 인 2차 다항식으로 변환하기 위해 PolynomialFeatures를 이용하여 변환
poly = PolynomialFeatures(degree=2)
poly.fit(X)
poly_ftr = poly.transform(X)
print('변환된 2차 다항식 계수 feature:\n', poly_ftr)
```

일차 단항식 계수 feature:
[[0 1]
[2 3]]
변환된 2차 다항식 계수 feature:
[[1. 0. 1. 0. 0. 1.]
[1. 2. 3. 4. 6. 9.]]

$$\begin{bmatrix} x_1 & x_2 \\ 0 & 1 \\ 2 & 3 \end{bmatrix}$$



	1	x_1	x_2	x_1^2	x_1x_2	x_2^2
첫 번째 샘플	1	0	1	0	0	1
두 번째 샘플	1	2	3	4	6	9

#5-5. 다항회귀

✓ 3차 다항회귀

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \epsilon$$

```
def polynomial_func(X):  
    y = 1 + 2*X[:,0] + 3*X[:,0]**2 + 4*X[:,1]**3  
    print(X[:, 0])  
    print(X[:, 1])  
    return y
```

```
X = np.arange(0,4).reshape(2,2)
```

```
print('일차 단항식 계수 feature: \n' ,X)
```

```
y = polynomial_func(X)  
print('삼차 다항식 결정값: \n' , y)
```

일차 단항식 계수 feature:

```
[[0 1]
```

```
 [2 3]]
```

```
[0 2]
```

```
[1 3]
```

삼차 다항식 결정값:

```
[ 5 125]
```

PolynomialFeatures(degree=3)

$[x_1, x_2]$



$[1 \quad x_1 \quad x_2 \quad x_1^2 \quad x_1x_2 \quad x_2^2 \quad x_1^3 \quad x_1^2x_2 \quad x_1x_2^2 \quad x_2^3]$

3 차 다항식 변환

```
poly_ftr = PolynomialFeatures(degree=3).fit_transform(X)
```

```
print('3차 다항식 계수 feature: \n',poly_ftr)
```

Linear Regression에 3차 다항식 계수 feature와 3차 다항식 결정값으로 학습 후 회귀 계수 확인

```
model = LinearRegression()
```

```
model.fit(poly_ftr,y)
```

```
print('Polynomial 회귀 계수\n' , np.round(model.coef_, 2))
```

```
print('Polynomial 회귀 Shape :', model.coef_.shape)
```

3차 다항식 계수 feature:

```
[[ 1.  0.  1.  0.  0.  1.  0.  0.  0.  1.]
```

```
 [ 1.  2.  3.  4.  6.  9.  8. 12. 18. 27.]]
```

Polynomial 회귀 계수

```
[0.  0.18 0.18 0.36 0.54 0.72 0.72 1.08 1.62 2.34]
```

Polynomial 회귀 Shape : (10,)

#5-5. 다항회귀

✓ 사이킷런 파이프라인(Pipeline)을 이용하여 3차 다항회귀 학습

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import Pipeline
import numpy as np
```

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \epsilon$$

```
def polynomial_func(X):
    y = 1 + 2*X[:,0] + 3*X[:,0]**2 + 4*X[:,1]**3
    return y

# Pipeline 객체로 Streamline 하게 Polynomial Feature변환과 Linear Regression을 연결
model = Pipeline([('poly', PolynomialFeatures(degree=3)),
                  ('linear', LinearRegression())])
X = np.arange(4).reshape(2,2)
y = polynomial_func(X)

model = model.fit(X, y)
print('Polynomial 회귀 계수\n', np.round(model.named_steps['linear'].coef_, 2))
```

```
Polynomial 회귀 계수
[0.  0.18 0.18 0.36 0.54 0.72 0.72 1.08 1.62 2.34]
```

```
# 3 차 다항식 변환
poly_ftr = PolynomialFeatures(degree=3).fit_transform(X)
print('3차 다항식 계수 feature: \n', poly_ftr)

# Linear Regression에 3차 다항식 계수 feature와 3차 다항식 결정값으로 학습 후 회귀 계수 확인
model = LinearRegression()
model.fit(poly_ftr, y)
print('Polynomial 회귀 계수\n', np.round(model.coef_, 2))
print('Polynomial 회귀 Shape :', model.coef_.shape)
```


#다항 회귀를 이용한 과소적합 및 과적합 이해

✓ 학습 데이터 생성

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score
%matplotlib inline

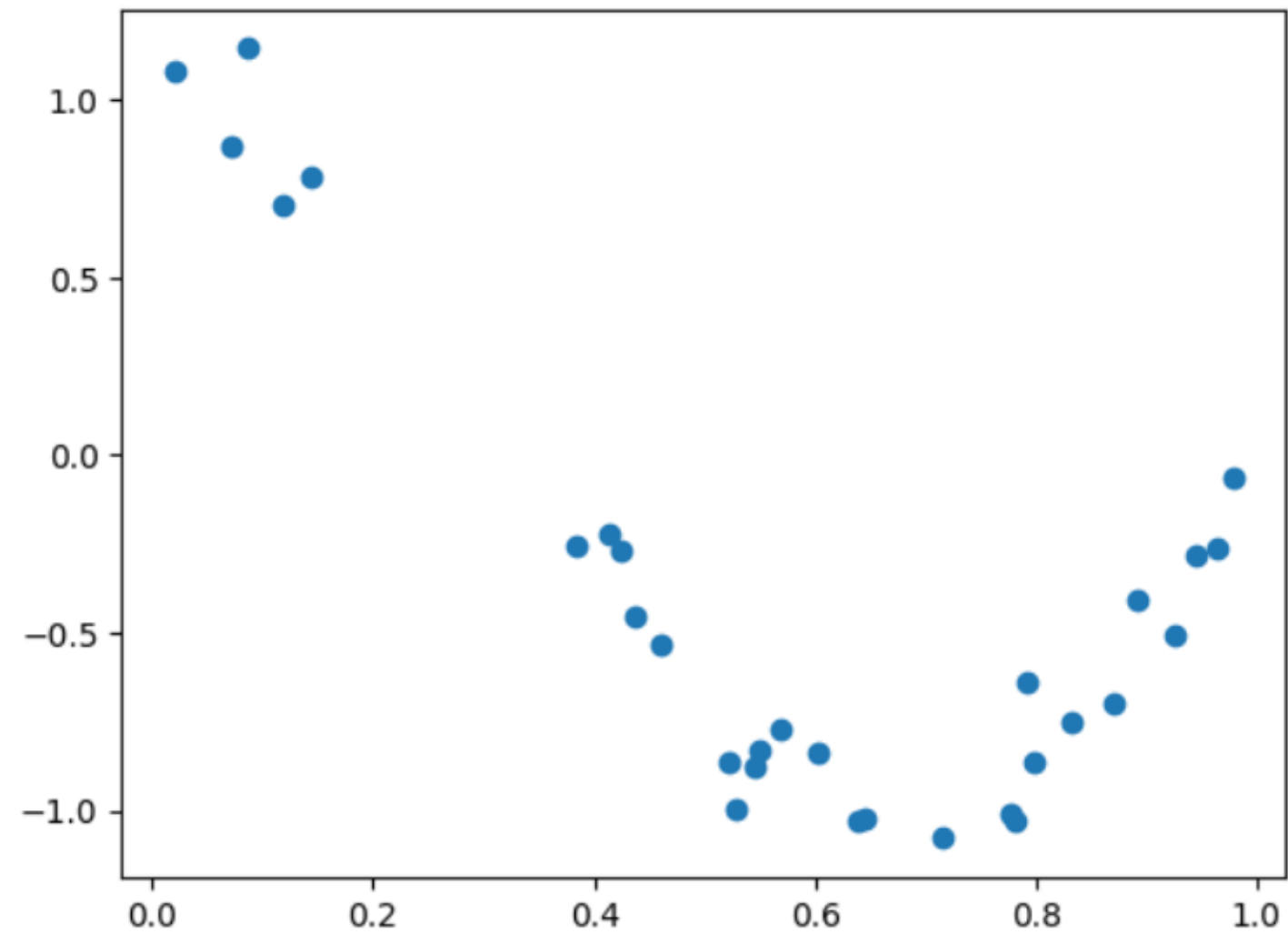
# 임의의 값으로 구성된 X값에 대해 코사인 변환 값을 반환.
def true_fun(X):
    return np.cos(1.5 * np.pi * X)

# X는 0부터 1까지 30개의 임의의 값을 순서대로 샘플링한 데이터입니다.
np.random.seed(0)
n_samples = 30
X = np.sort(np.random.rand(n_samples))

# y 값은 코사인 기반의 true_fun()에서 약간의 노이즈 변동 값을 더한 값입니다.
y = true_fun(X) + np.random.randn(n_samples) * 0.1

plt.scatter(X, y)
```

📌 X-y : 코사인 함수 + noise > 비선형 함수



#다항 회귀를 이용한 과소적합 및 과적합 이해

✓ 1차, 4차, 15차 다항식으로 변경하며 예측 성능 평가

```
plt.figure(figsize=(14, 5))
degrees = [1, 4, 15]

# 다항 회귀의 차수(degree)를 1, 4, 15로 각각 변화시키면서 비교합니다.
for i in range(len(degrees)):
    ax = plt.subplot(1, len(degrees), i + 1)
    plt.setp(ax, xticks=(), yticks=())

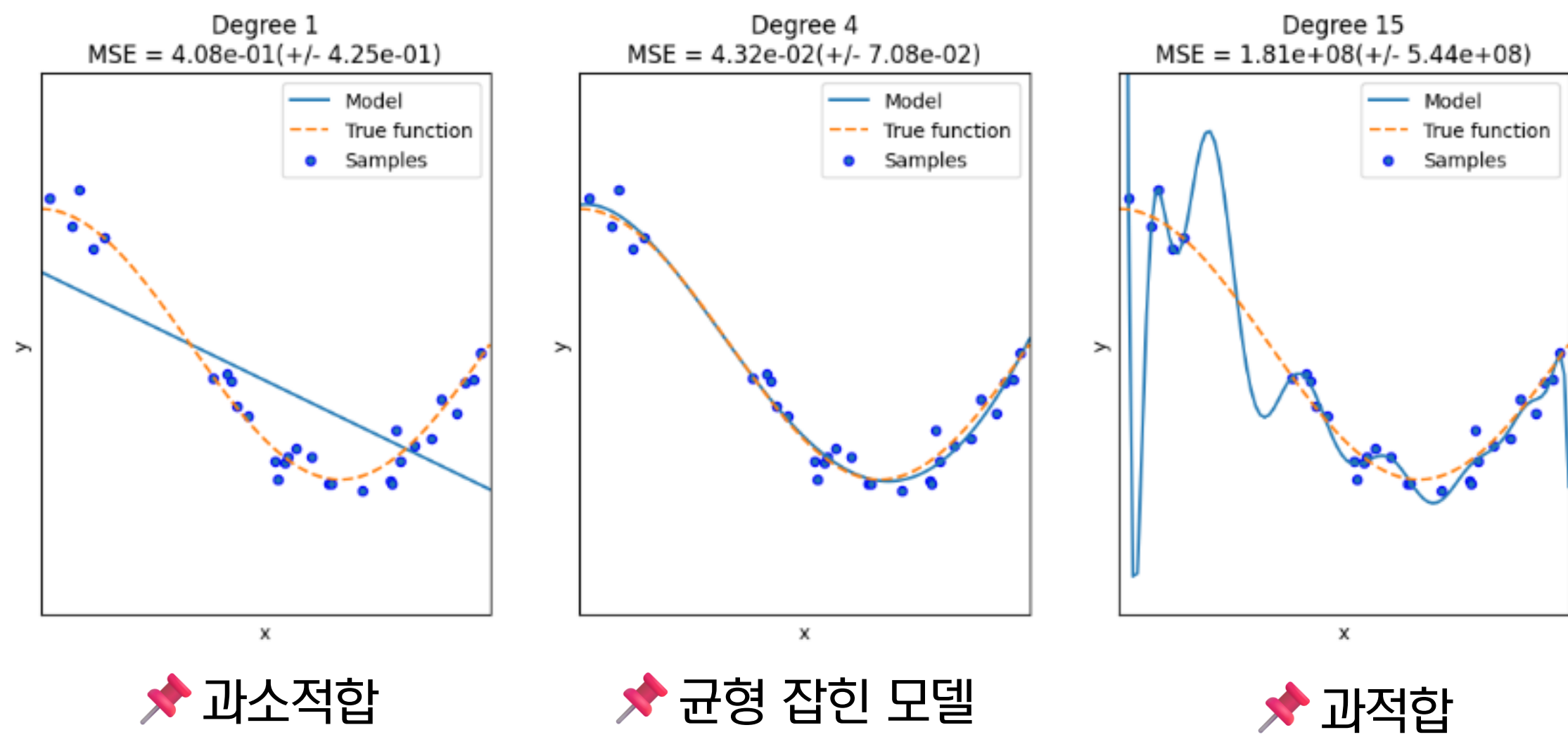
    # 개별 degree별로 Polynomial 변환합니다.
    polynomial_features = PolynomialFeatures(degree=degrees[i], include_bias=False)
    linear_regression = LinearRegression()
    pipeline = Pipeline([("polynomial_features", polynomial_features),
                          ("linear_regression", linear_regression)])
    pipeline.fit(X.reshape(-1, 1), y)

    # 교차 검증으로 다항 회귀를 평가합니다.
    scores = cross_val_score(pipeline, X.reshape(-1, 1), y, scoring="neg_mean_squared_error", cv=10)
    # Pipeline을 구성하는 세부 객체를 접근하는 named_steps['객체명']을 이용해 회귀계수 추출
    coefficients = pipeline.named_steps['linear_regression'].coef_
    print('\nDegree {0} 회귀 계수는 {1} 입니다.'.format(degrees[i], np.round(coefficients, 2)))
    print('Degree {0} MSE 는 {1} 입니다.'.format(degrees[i], -1*np.mean(scores)))

    # 0 부터 1까지 테스트 데이터 세트를 100개로 나눠 예측을 수행합니다.
    # 테스트 데이터 세트에 회귀 예측을 수행하고 예측 곡선과 실제 곡선을 그려서 비교합니다.
    X_test = np.linspace(0, 1, 100)
    # 예측값 곡선
    plt.plot(X_test, pipeline.predict(X_test[:, np.newaxis]), label="Model")
    # 실제 값 곡선
    plt.plot(X_test, true_fun(X_test), '--', label="True function")
    plt.scatter(X, y, edgecolor='b', s=20, label="Samples")
    plt.xlabel("x"); plt.ylabel("y"); plt.xlim((0, 1)); plt.ylim((-2, 2)); plt.legend(loc="best")
    plt.title("Degree {0} MSE = {:.2e} (+/- {:.2e})".format(degrees[i], -scores.mean(), scores.std()))
```

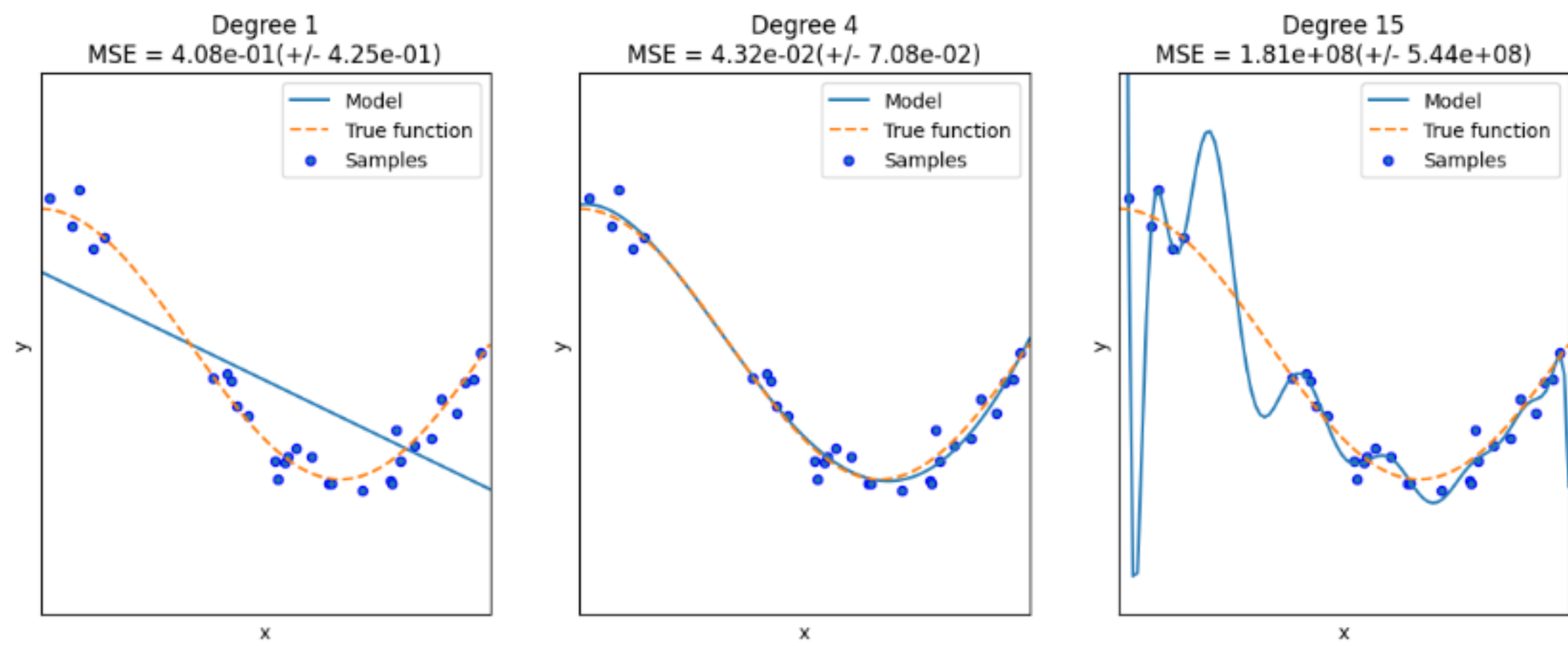
#다항 회귀를 이용한 과소적합 및 과적합 이해

✓ 성능 평가 결과



편향 분산 트레이드 오프(Bias-Variance Trade off)

✓ 성능 평가 결과



📌 한 방향으로 치우침

📌 변동성이 큼

편향 분산 트레이드 오프(Bias-Variance Trade off)

✓ 편향과 분산은 시소와 같은 관계

$$\mathbb{E}[(\hat{f}(x) - f(x))^2] = \underbrace{\left(\mathbb{E}[\hat{f}(x)] - f(x)\right)^2}_{\text{편향}} + \underbrace{\mathbb{E}\left[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2\right]}_{\text{분산}} + \sigma^2$$

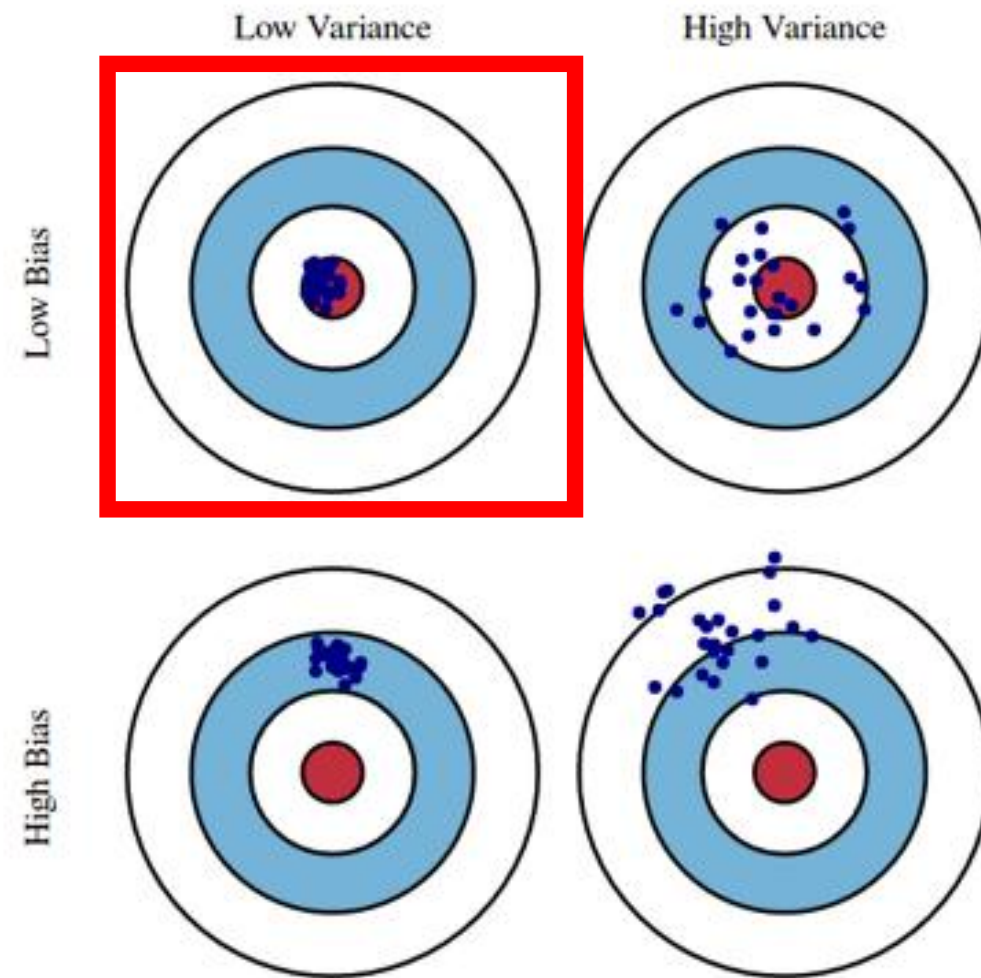
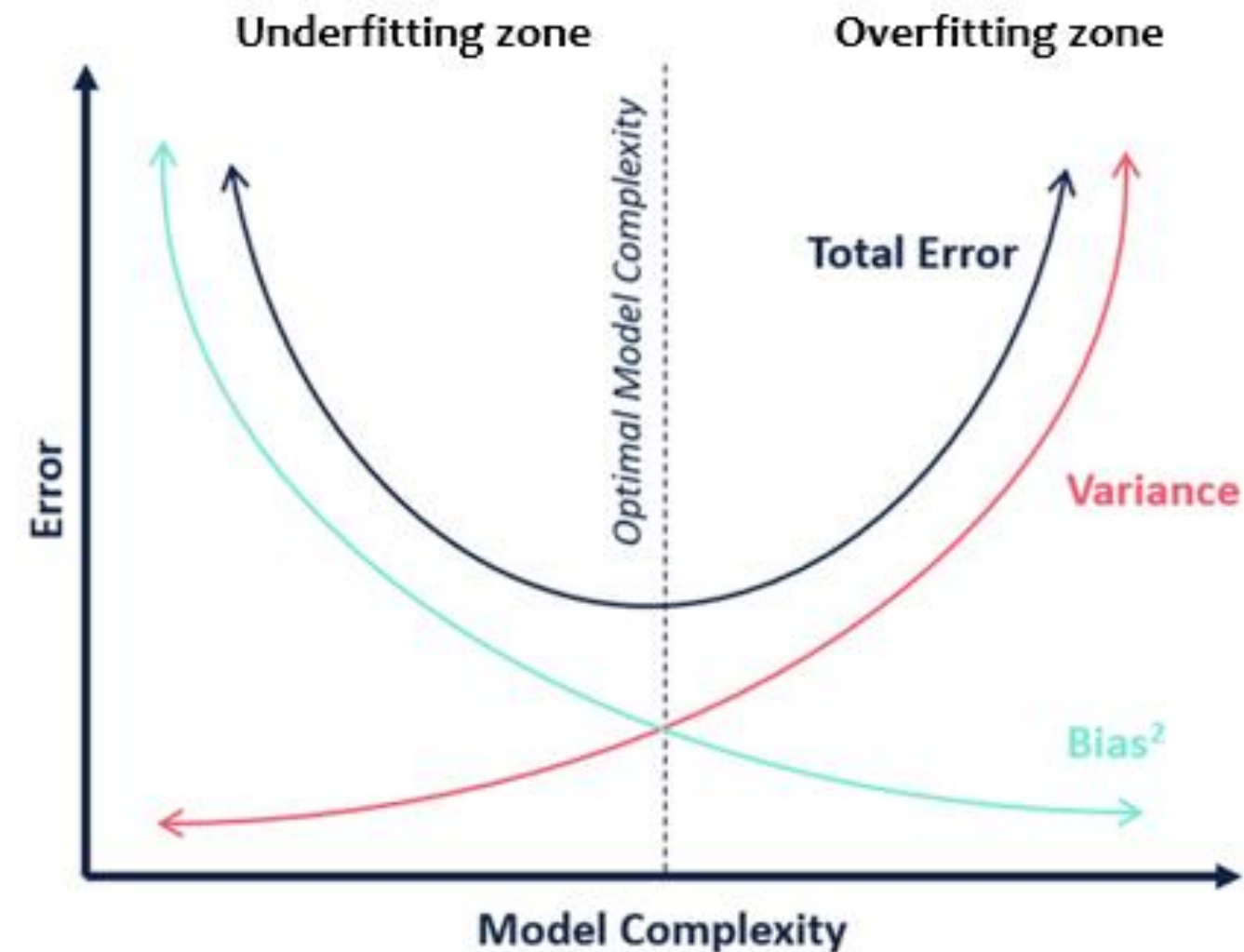


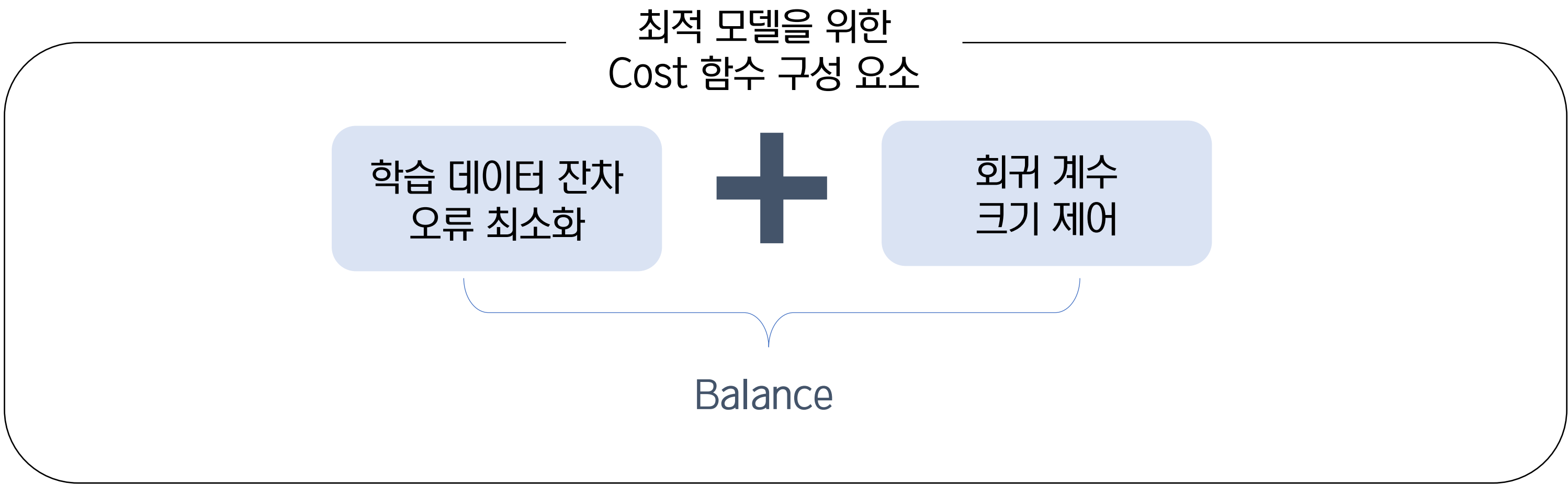
Fig. 1 Graphical illustration of bias and variance.



5.6 규제 선형 회귀



규제 선형 모델 : 규제의 필요성



비용 함수 목표

$$\text{Min}(RSS(W) + \alpha * ||W||_2^2)$$

규제 선형 모델 : 규제 원리

비용 함수 목표

$$\text{Min}(RSS(W) + \alpha * ||W||_2^2)$$

감소 ← α → 증가

$$\text{Min}(RSS(W) + \alpha * ||W||_2^2)$$

$\text{Min}(RSS(W) + \alpha * ||W||_2^2)$

- ⇒ 계수 크기 최소화 필요
- ⇒ 계수 크기 증가 억제

규제 선형 모델 : L1 & L2 규제

비용 함수 목표

$$\text{Min}(RSS(W) + \alpha * ||W||_2^2)$$

$$||W||_2^2 = w_1^2 + w_2^2 + \dots$$

L2 규제

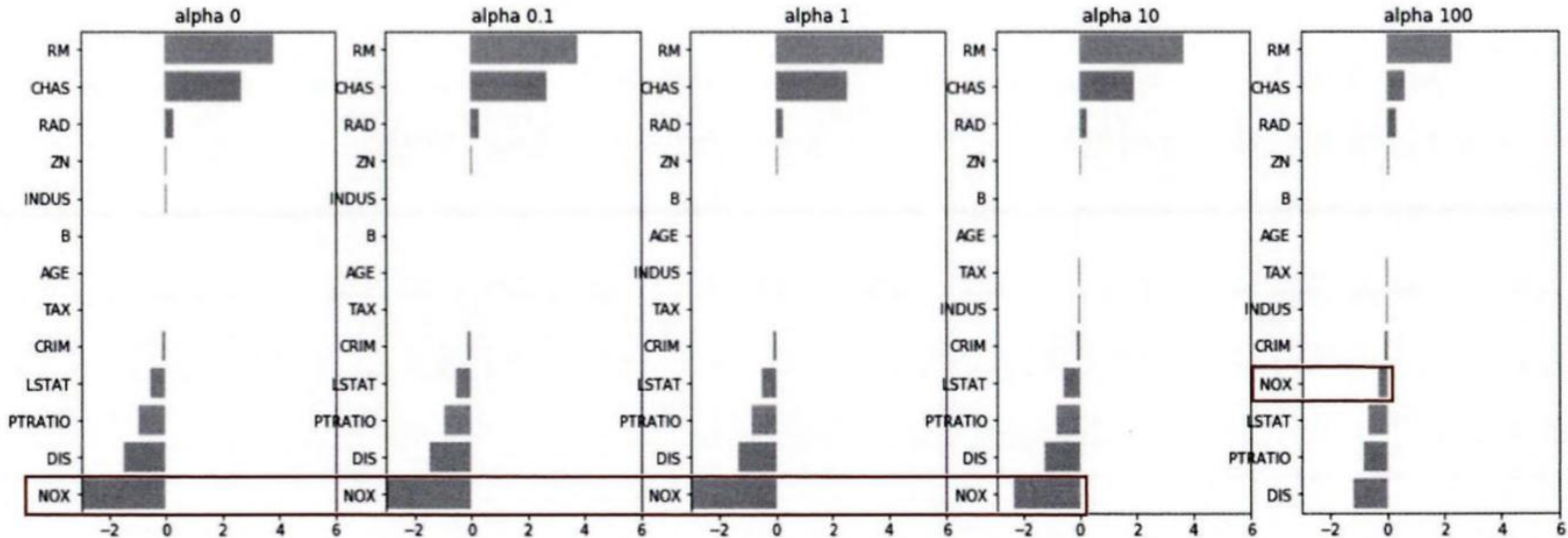
$$||W||_1 = |w_1| + |w_2| + \dots$$

L1 규제

규제 선형 모델 : 릿지 회귀

```
class Ridge(sklearn.base.MultiOutputMixin, sklearn.base.RegressorMixin, _BaseRegressorMixin):  
    | Ridge(alpha=1.0, *, fit_intercept=True, copy_X=True, max_iter=None, tol=0.0001, verbose=0)
```

【Output】



규제 선형 모델 : 릿지 회귀

	alpha:0	alpha:0.1	alpha:1	alpha:10	alpha:100
RM	3.809865	3.818233	3.854000	3.702272	2.334536
CHAS	2.686734	2.670019	2.552393	1.952021	0.638335
RAD	0.306049	0.303515	0.290142	0.279596	0.315358
ZN	0.046420	0.046572	0.047443	0.049579	0.054496
INDUS	0.020559	0.015999	-0.008805	-0.042962	-0.052826
B	0.009312	0.009368	0.009673	0.010037	0.009393
AGE	0.000692	-0.000269	-0.005415	-0.010707	0.001212
TAX	-0.012335	-0.012421	-0.012912	-0.013993	-0.015856
CRIM	-0.108011	-0.107474	-0.104595	-0.101435	-0.102202
LSTAT	-0.524758	-0.525966	-0.533343	-0.559366	-0.660764
PTRATIO	-0.952747	-0.940759	-0.876074	-0.797945	-0.829218
DIS	-1.475567	-1.459626	-1.372654	-1.248808	-1.153390
NOX	-17.766611	-16.684645	-10.777015	-2.371619	-0.262847

규제 선형 모델 : 라쏘 회귀

	alpha:0.07	alpha:0.1	alpha:0.5	alpha:1	alpha:3
RM	3.789725	3.703202	2.498212	0.949811	0.000000
CHAS	1.434343	0.955190	0.000000	0.000000	0.000000
RAD	0.270936	0.274707	0.277451	0.264206	0.061864
ZN	0.049059	0.049211	0.049544	0.049165	0.037231
B	0.010248	0.010249	0.009469	0.008247	0.006510
NOX	-0.000000	-0.000000	-0.000000	-0.000000	0.000000
AGE	-0.011706	-0.010037	0.003604	0.020910	0.042495
TAX	-0.014290	-0.014570	-0.015442	-0.015212	-0.008602
INDUS	-0.042120	-0.036619	-0.005253	-0.000000	-0.000000
CRIM	-0.098193	-0.097894	-0.083289	-0.063437	-0.000000
LSTAT	-0.560431	-0.568769	-0.656290	-0.761115	-0.807679
PTRATIO	-0.765107	-0.770654	-0.758752	-0.722966	-0.265072
DIS	-1.176583	-1.160538	-0.936605	-0.668790	-0.000000

규제 선형 모델 : 엘라스틱넷 회귀

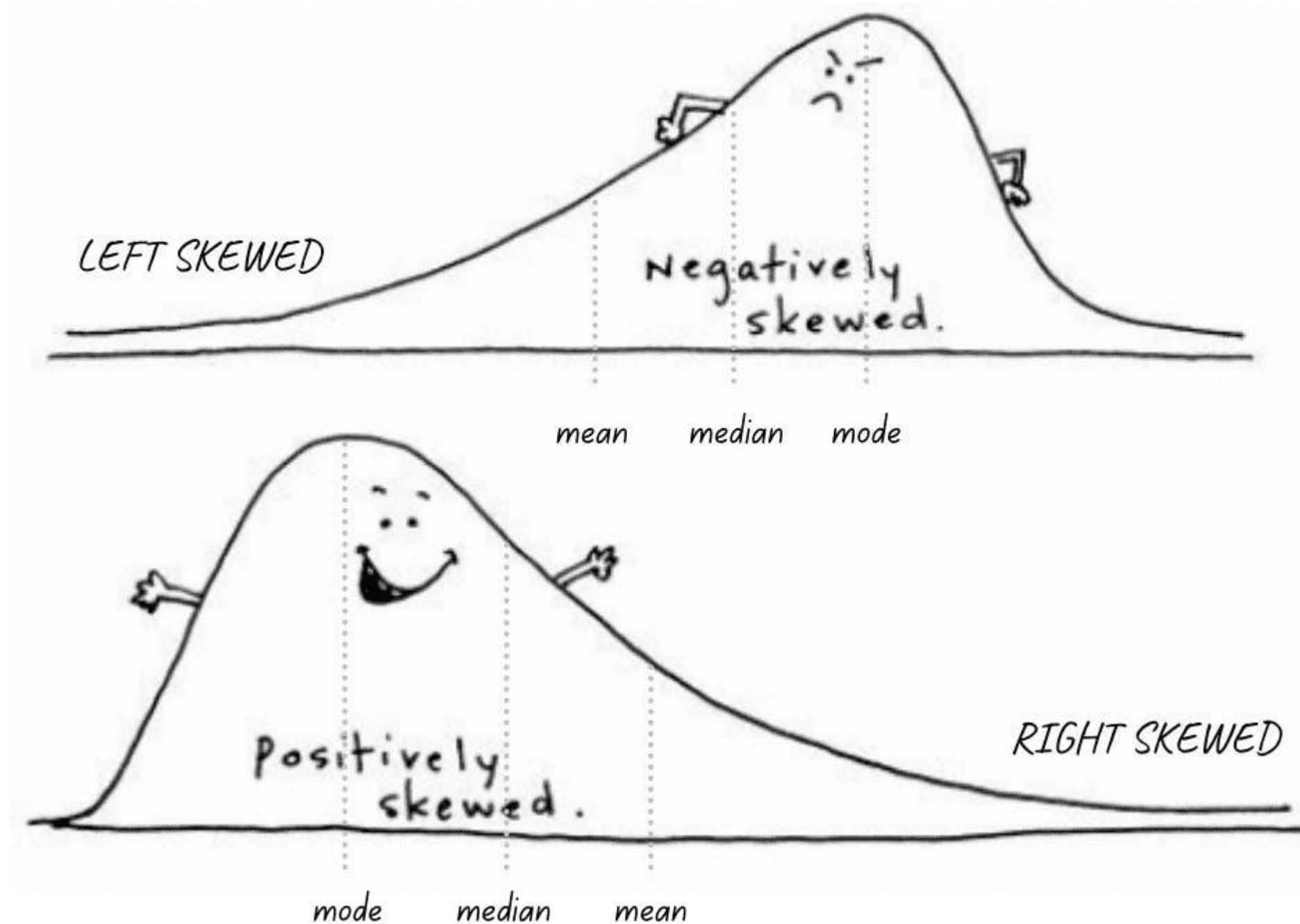
비용 함수 목표

$$\text{Min}(RSS(W) + \alpha_1 * ||W||_2^2 + \alpha_2 * ||W||_1)$$

	alpha:0.07	alpha:0.1	alpha:0.5	alpha:1	alpha:3
RM	3.574162	3.414154	1.918419	0.938789	0.000000
CHAS	1.330724	0.979706	0.000000	0.000000	0.000000
RAD	0.278880	0.283443	0.300761	0.289299	0.146846
ZN	0.050107	0.050617	0.052878	0.052136	0.038268
B	0.010122	0.010067	0.009114	0.008320	0.007020
AGE	-0.010116	-0.008276	0.007760	0.020348	0.043446
TAX	-0.014522	-0.014814	-0.016046	-0.016218	-0.011417
INDUS	-0.044855	-0.042719	-0.023252	-0.000000	-0.000000
CRIM	-0.099468	-0.099213	-0.089070	-0.073577	-0.019058
NOX	-0.175072	-0.000000	-0.000000	-0.000000	-0.000000
LSTAT	-0.574822	-0.587702	-0.693861	-0.760457	-0.800368
PTRATIO	-0.779498	-0.784725	-0.790969	-0.738672	-0.423065
DIS	-1.189438	-1.173647	-0.975902	-0.725174	-0.031208

선형 회귀 모델을 위한 데이터 변환

← Left or Right Skewed? Follow the tail. →

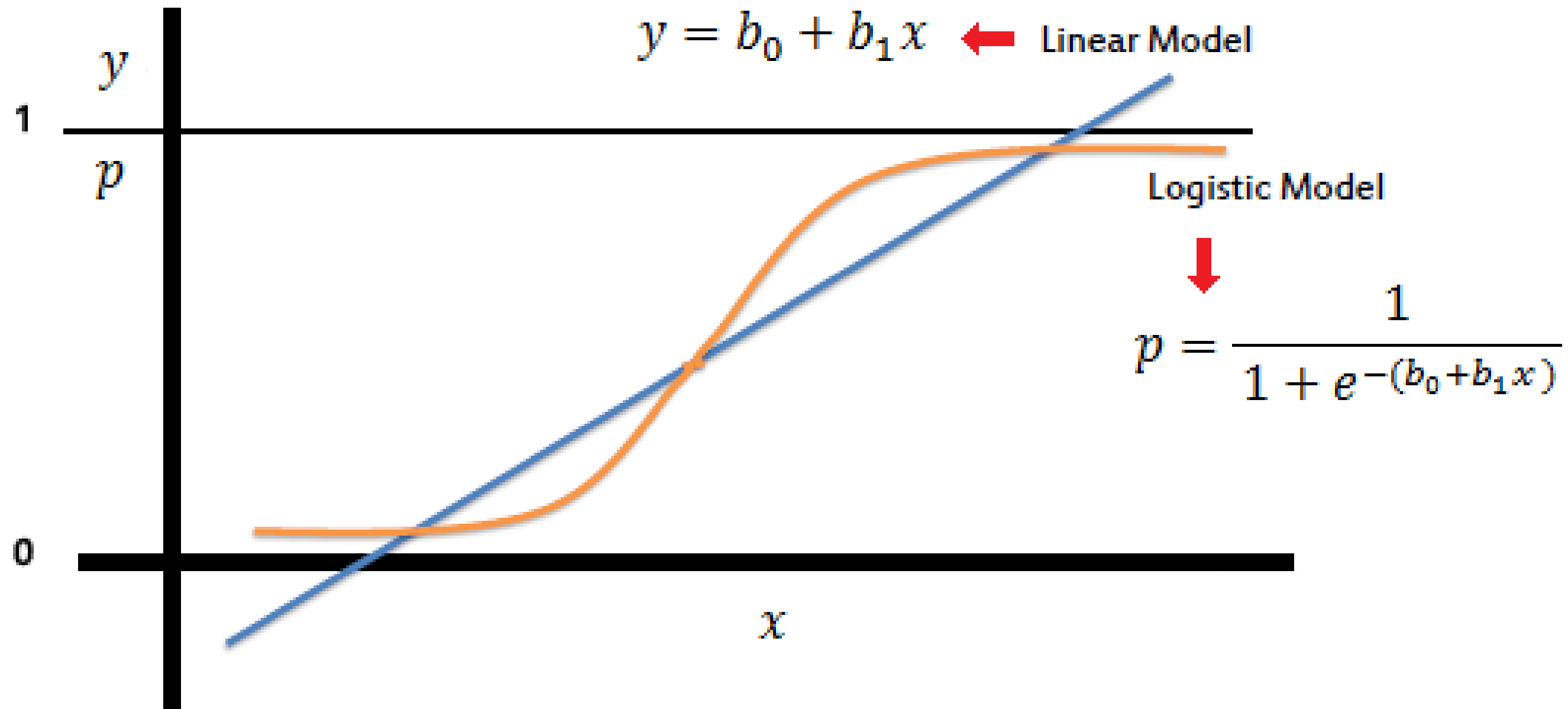


- ① StandardScaler
MinMaxScaler
- ② 다항 특성 적용
- ③ Log 변환

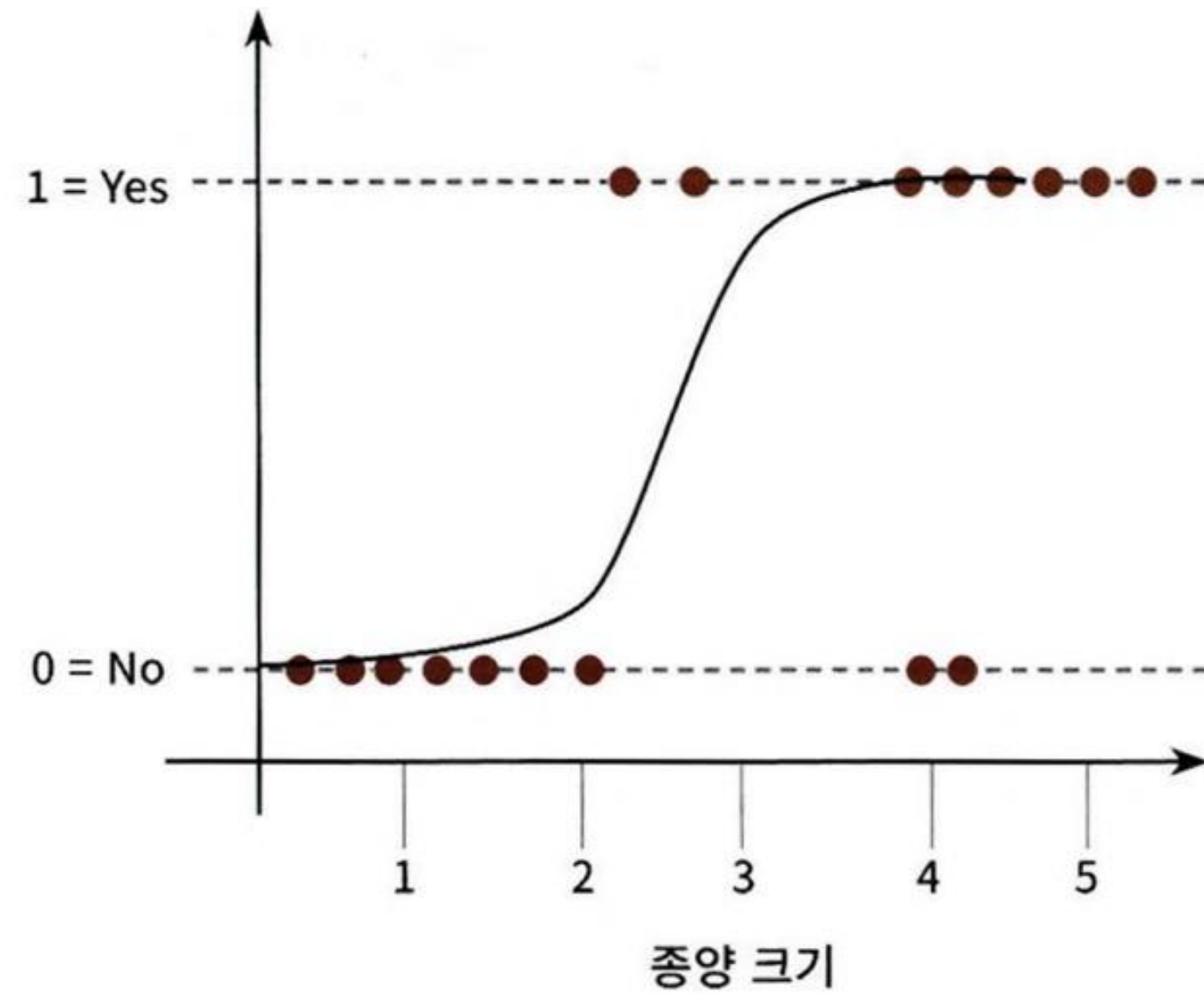
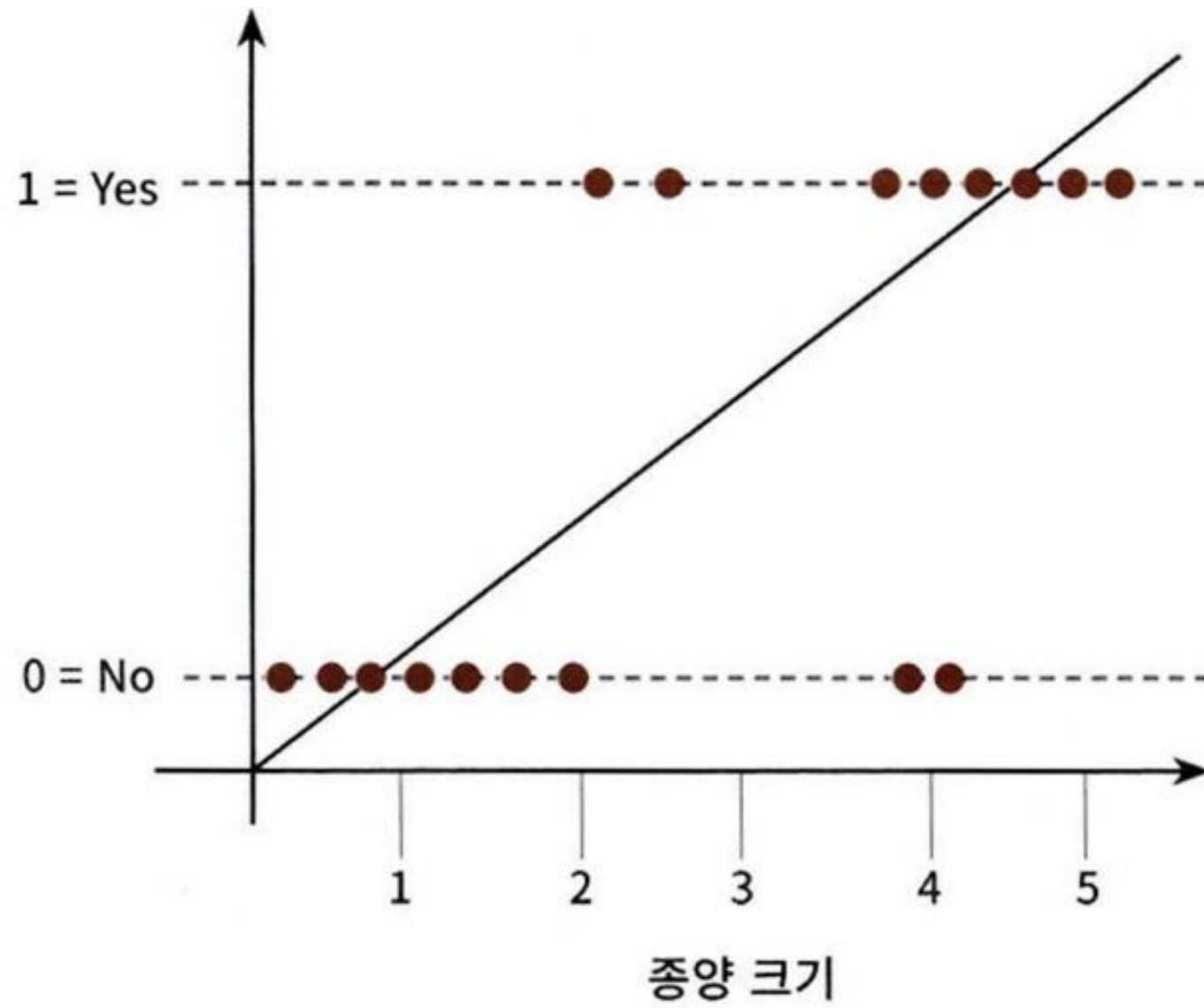
5.7 로지스틱 회귀



로지스틱 회귀



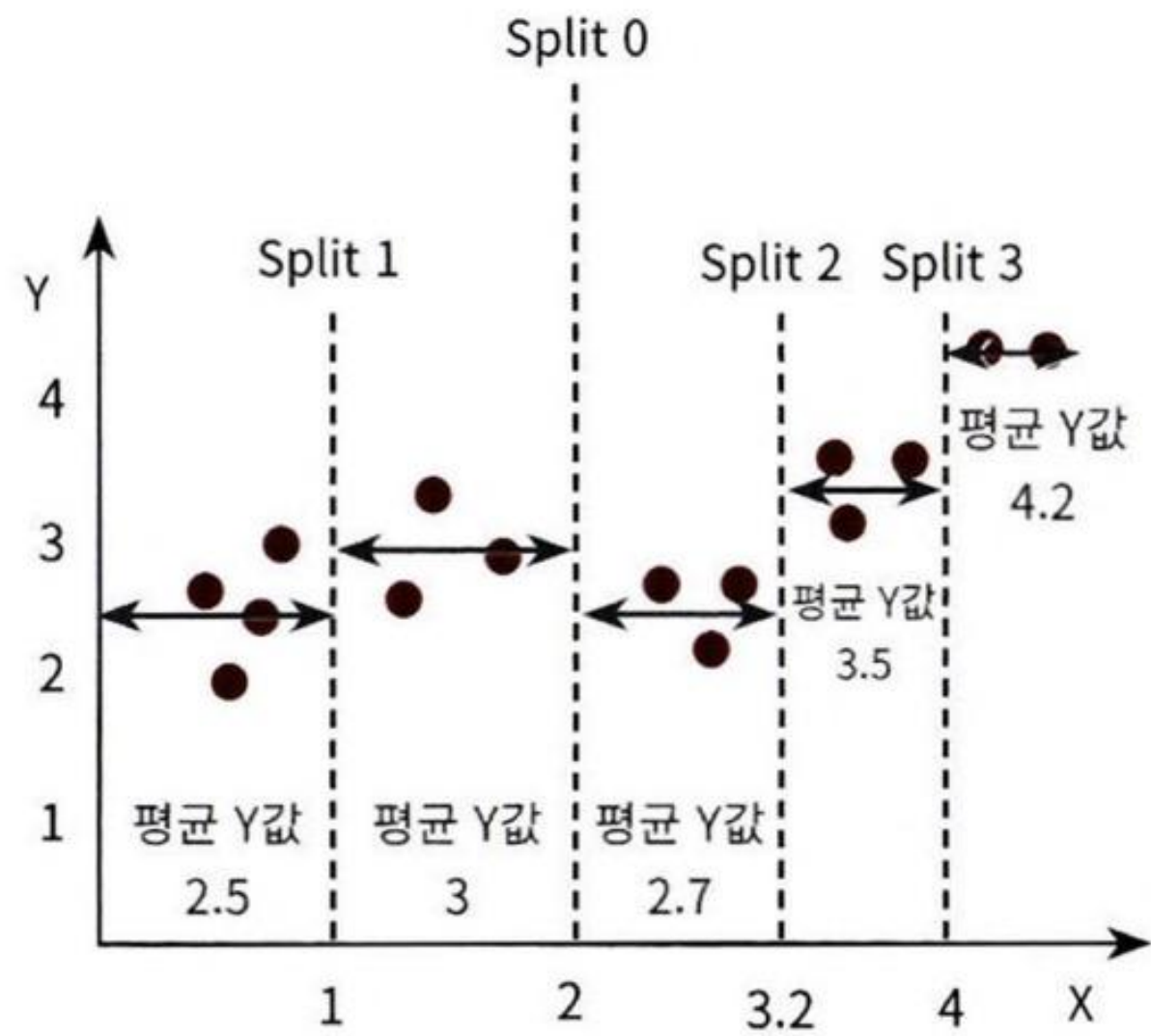
로지스틱 회귀



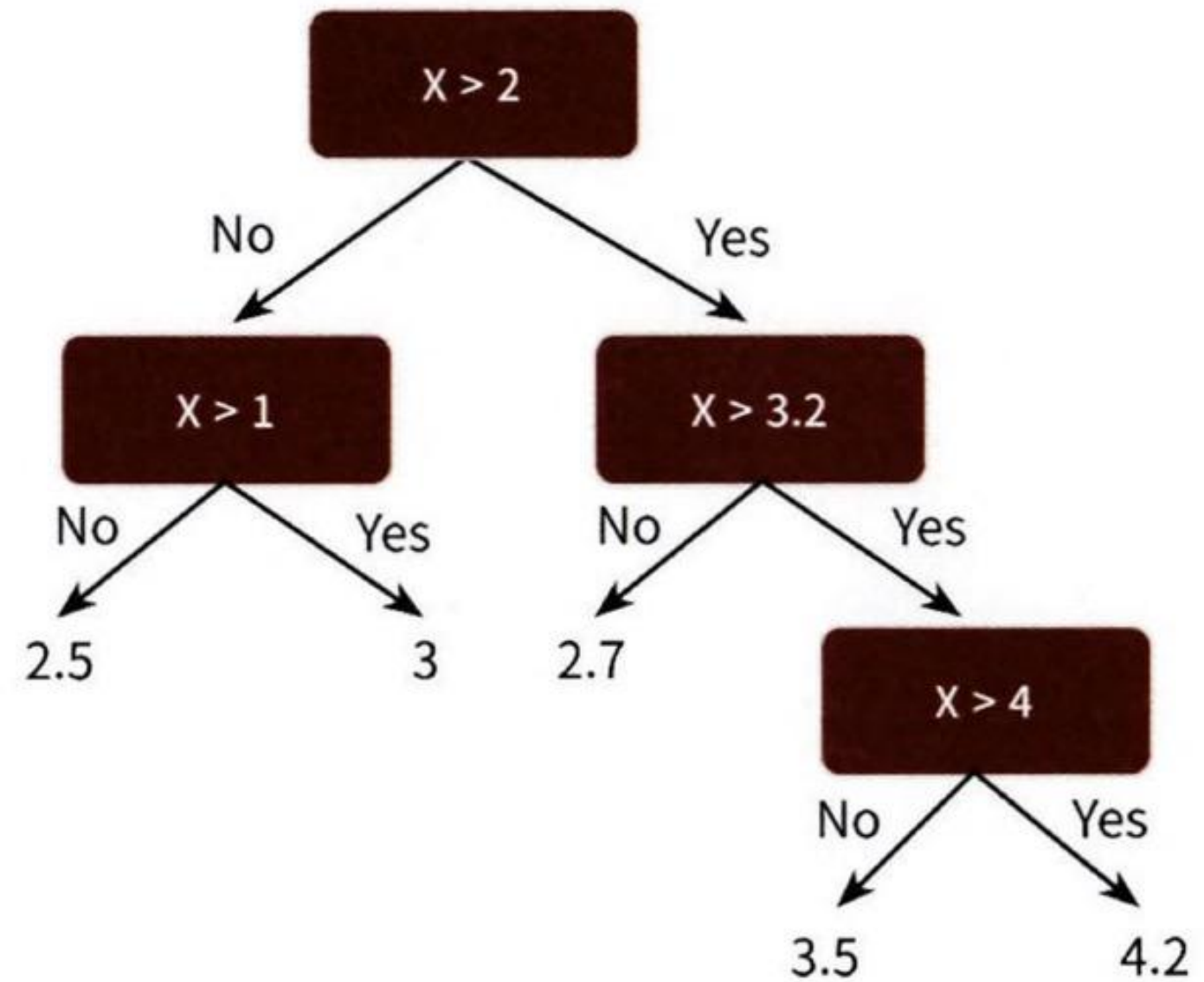
5.8 회귀 트리



회귀 트리



평균 Y값으로
예측 값 부여



회귀 트리

알고리즘	회귀 Estimator 클래스	분류 Estimator 클래스
Decision Tree	DecisionTreeRegressor	DecisionTreeClassifier
Gradient Boosting	GradientBoostingRegressor	GradientBoostingClassifier
XGBoost	XGBRegressor	XGBClassifier
LightGBM	LGBMRegressor	LGBMClassifier

THANK YOU

